

MLPy Workshop 4

February 8, 2026

1 Week 4 - Regression and Model Evaluation

Student Name 1, Student Name 2

1.1 Aims

By the end of this notebook you will be able to

- fit a linear regression
- understand the basics of polynomial regression
- understand how to evaluate and compare models and select tuning parameters with training, validation, and testing.

1. Problem Definition and Setup
2. Exploratory Data Analysis
3. Least Squares Estimation
4. Regression using scikit-Learn
5. Polynomial Regression

During workshops, you will complete the worksheets together in teams of 2-3, using **pair programming**. From this week onwards, the worksheets will no longer contain cues to switch roles between driver and navigator; this should occur approximately every 15 minutes and should be more natural after the first weeks. When completing worksheets:

- You will have tasks tagged by (CORE) and (EXTRA).
- Your primary aim is to complete the (CORE) components during the WS session, afterwards you can try to complete the (EXTRA) tasks for your self-learning process.

Instructions for submitting your workshops can be found at the end of worksheet. As a reminder, you must submit a pdf of your notebook on Learn by 16:00 PM on the Friday of the week the workshop was given.

2 Problem Definition and Setup

2.1 Packages

First, let's load the packages you will need for this workshop.

```
[1]: # Data libraries
import pandas as pd
import numpy as np

# Plotting libraries
import matplotlib.pyplot as plt
import seaborn as sns

# sklearn modules and some other will be added later
import sklearn
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import GridSearchCV, KFold
```

2.2 User Defined Helper Functions

Below are two helper functions we will be using in this workshop. You can modify or create your own if you think it is useful or simply use already available functions within `sklearn`.

- `model_fit()`: Returns the mean squared error, root mean squared error and R^2 value of a fitted model based on the provided X and y values with plotting as add-on.
- `compute_std_error()`: Computes the standard error of the coefficients, with plotting as add-on.

```
[2]: def model_fit(m, X, y, plot = False):
    """Returns the mean squared error, root mean squared error and  $R^2$  value of
    a fitted model based
    on provided X and y values.

    Args:
        m: sklearn model object
        X: model matrix to use for prediction
        y: outcome vector to use to calculating rmse and residuals
        plot: boolean value, should residual plots be shown
    """

    y_hat = m.predict(X)
    MSE = mean_squared_error(y, y_hat)
    RMSE = np.sqrt(mean_squared_error(y, y_hat))
    Rsqr = r2_score(y, y_hat)

    Metrics = (round(MSE, 4), round(RMSE, 4), round(Rsqr, 4))
```

```

res = pd.DataFrame(
    data = {'y': y, 'y_hat': y_hat, 'resid': y - y_hat}
)

if plot:
    plt.figure(figsize=(12, 6))

    plt.subplot(121)
    sns.lineplot(x='y', y='y_hat', color="grey", data = pd.
↳DataFrame(data={'y': [min(y),max(y)], 'y_hat': [min(y),max(y)]}))
    sns.scatterplot(x='y', y='y_hat', data=res).set_title("Observed vs_
↳Fitted values")

    plt.subplot(122)
    sns.scatterplot(x='y_hat', y='resid', data=res).set_title("Fitted_
↳values vs Residuals")
    plt.hlines(y=0, xmin=np.min(y), xmax=np.max(y), linestyle='dashed',_
↳alpha=0.3, colors="black")

    plt.subplots_adjust(left=0.0)

    plt.suptitle("Model (MSE, RMSE, Rsq) = " + str(Metrics), fontsize=14)
    plt.show()

return MSE, RMSE, Rsqr

```

```

[3]: def compute_std_error(m, X, y, plot = False, feature_names = None, figsize =_
↳(5,5)):
    """Returns the standard errors of the model coefficients for a fitted_
↳linear model.

    Args:
        m: sklearn LinearRegression model object or pipeline with_
↳LinearRegression as final step
        X: training input matrix
        y: training outcome vector
        plot: boolean value, should coefficients be plotted with error bars
        feature_names: list of feature names to use in the plot
    """

    y_hat = m.predict(X)
    # Define feature matrix with intercept
    X_ = np.hstack([np.ones((X.shape[0], 1)), X])
    # If m is a pipeline, transform X accordingly
    if isinstance(m, sklearn.pipeline.Pipeline):
        X_ = np.hstack([np.ones((X.shape[0], 1)), m[:-1].transform(X)])

```

```

sigmsq = np.sum((y - y_hat)**2) / (X_.shape[0] - X_.shape[1]-1)
cov_mat = sigmsq * np.linalg.inv(X_.T@X_)
sd_er = np.sqrt(np.diag(cov_mat))

if plot:
    if feature_names is None:
        feature_names = m[:-1].get_feature_names_out() if isinstance(m,
↳sklearn.pipeline.Pipeline) else m.feature_names_in_

    coef = m[-1].coef_ if isinstance(m, sklearn.pipeline.Pipeline) else m.
↳coef_
    # Create data frame for plotting
    df = pd.DataFrame({'feature': feature_names, 'coef': coef, 'std_err': 1.
↳96*sd_er[1:]})
    df.sort_values ("coef", ascending=False)

    plt.figure(figsize=figsize)
    plt.bar(df['feature'], df['coef'])
    plt.errorbar(df['feature'], df['coef'], yerr=df['std_err'], fmt="o",
↳color="r")
    #sns.barplot(data=df, y='feature', x='coef')
    plt.title('Coefficient Estimates with 95% Confidence Intervals')
    plt.xlabel('Features')
    plt.ylabel('Coefficient Value')
    plt.xticks(rotation=90)
    plt.grid()
    plt.show()

return sd_er, cov_mat

```

2.3 Data

To begin, we will examine `insurance.csv` data set on the medical costs which comes from the [Medical Cost Personal dataset](#). Our goal is to model the yearly medical charges of an individual using some combination of the other features in the data. The included columns are as follows:

- **charges** - yearly medical charges in USD
- **age** - the individuals age
- **sex** - the individuals sex, either "male" or "female"
- **bmi** - the body mass index of the individual
- **children** - the number of dependent children the individual has
- **smoker** - a factor with levels "yes", the individual is a smoker and "no", the individual is not a smoker

We read the data into python using pandas.

```

[4]: df_insurance = pd.read_csv("insurance.csv")
df_insurance.head()

```

```
[4]:   age      sex      bmi  children smoker      charges
0    19  female  27.900         0     yes  16884.92400
1    18   male  33.770         1     no   1725.55230
2    28   male  33.000         3     no   4449.46200
3    33   male  22.705         0     no  21984.47061
4    32   male  28.880         0     no   3866.85520
```

3 Exploratory Data Analysis

Before modelling, we will start with EDA to gain an understanding of the data, through descriptive statistics and visualizations.

3.0.1 Exercise 1 (CORE)

- Examine the data structure using `.info()` and look at the descriptive statistics using `.describe()`. What are the types of variables in the data set? Are there any missing values?
- Create a pairs plot of the data (make sure to color by `smoker` to visualize its influence). Describe any relationships you observe in the data. To better visualize the relationship between `children` and `charges`, create a violin plot (since `children` only takes a small number of integer values, many points are overlaid in the scatterplot and making visualization difficult).

Hint

- You can use `sns.pairplot` and `sns.violinplot` with the `hue` argument to color by `smoker`.

```
[5]: #Part a

# For general info
df_insurance.info()

# For the summary statistics
df_insurance.describe()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1338 entries, 0 to 1337
Data columns (total 6 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   age         1338 non-null   int64
 1   sex         1338 non-null   object
 2   bmi         1338 non-null   float64
 3   children    1338 non-null   int64
 4   smoker      1338 non-null   object
 5   charges     1338 non-null   float64
dtypes: float64(2), int64(2), object(2)
memory usage: 62.8+ KB
```

```
[5]:
```

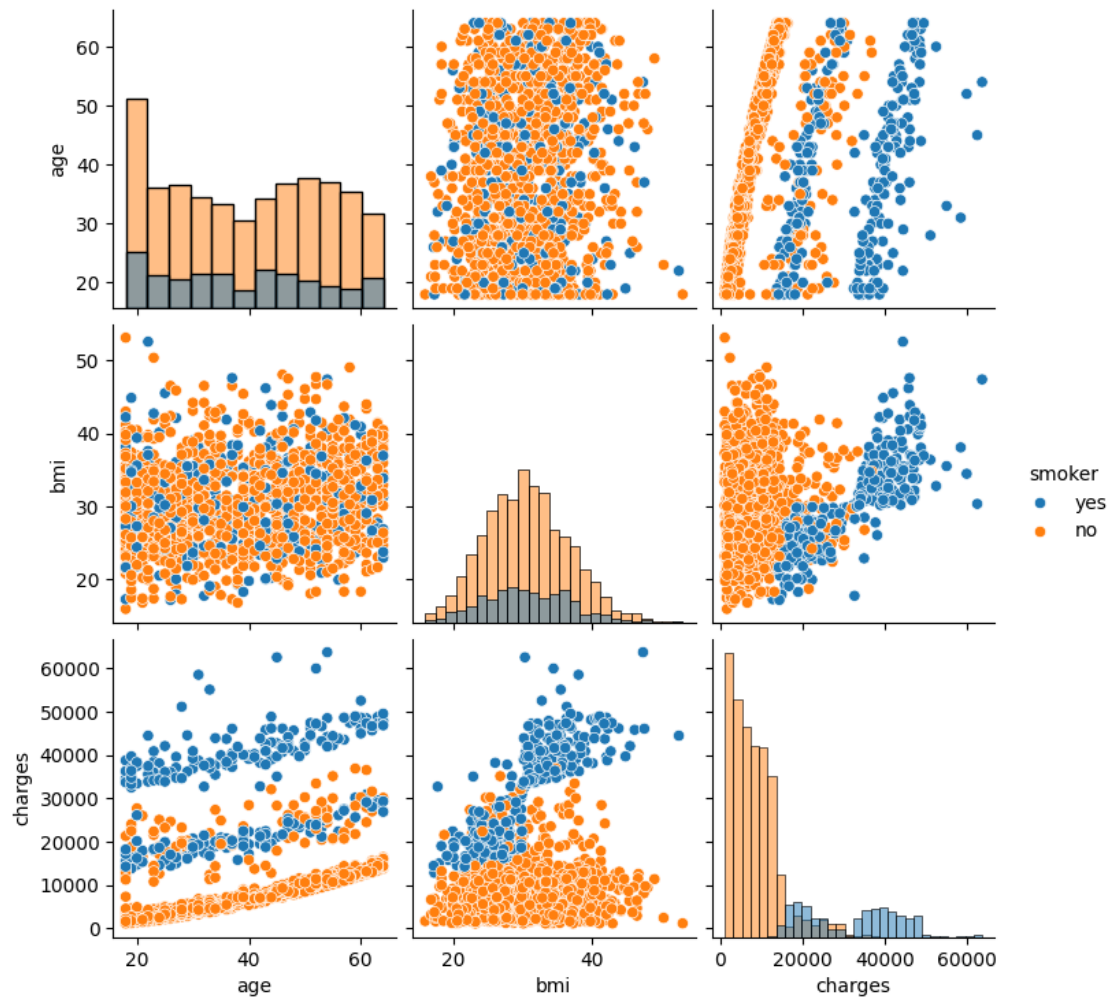
	age	bmi	children	charges
count	1338.000000	1338.000000	1338.000000	1338.000000
mean	39.207025	30.663397	1.094918	13270.422265
std	14.049960	6.098187	1.205493	12110.011237
min	18.000000	15.960000	0.000000	1121.873900
25%	27.000000	26.296250	0.000000	4740.287150
50%	39.000000	30.400000	1.000000	9382.033000
75%	51.000000	34.693750	2.000000	16639.912515
max	64.000000	53.130000	5.000000	63770.428010

The target variable **charges** is numeric, with both a large mean and scale; **bmi** is also a numeric variable, while **age** and **children** are both integers. There are two binary categorical variables that we will need to encode, which are **sex** and **smoker**. There are 1338 entries with no missing values.

```
[6]: # Part b

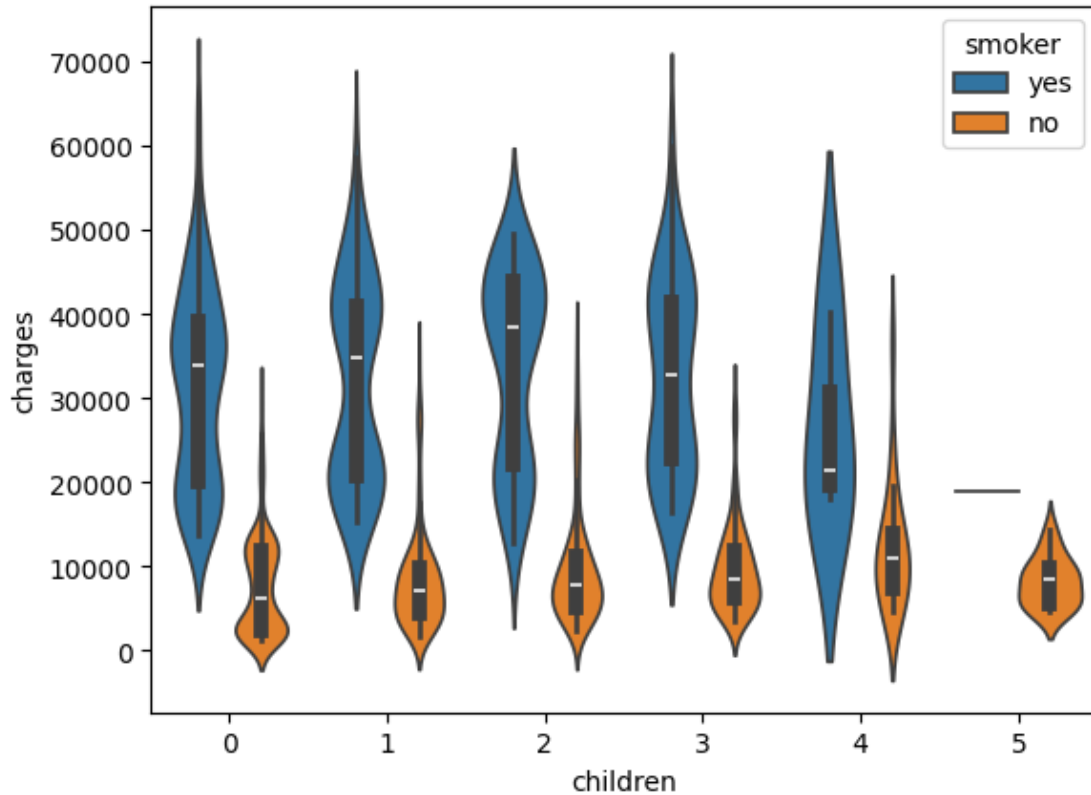
# # Plotting the data set
# sns.pairplot(data = df_insurance, hue = "smoker", diag_kind='hist')
# plt.show()

# Many points are plotted on top of each other for children, let's remove it
↳ from the pairplot
sns.pairplot(data = df_insurance, hue = "smoker", diag_kind='hist',
↳ vars=['age', 'bmi', 'charges'])
plt.show()
```



```
[7]: # Violin plot for discrete variables
sns.violinplot(data=df_insurance, x='children', y='charges', hue='smoker')
plt.show()

# # Or a strip plot
# sns.stripplot(data=df_insurance, x='children', y='charges', hue='smoker')
# plt.show()
```



From the pairs plot, we observe that:

- **bmi** and **charges** as well as **age** and **charges** pairs seem to exhibit possible linear relationships.
- There is a distinct difference in the **charges** between smokers and non smokers, with overall smaller charges for non-smokers.
- Our response variable **charges** seems to be skewed.
- There does not appear to be an evident relationship between **children** and **charges**.

3.1 Creating a Train-Test Set

Before modelling, we will first split the data into the train and test sets. This ensures that that we do not violate one of the golden rules of machine learning: never use the test set for training. As EDA can help guide the choice and form of model, we ideally want to split the data before EDA, to avoid peeking at the test data too much during this phase. However, in practice, we may need to investigate the entire data during EDA to get a better idea on how to handle issues such as missingness, categorical data (and rare categories), incorrect data, etc.

There are lots of ways of creating a test set. We will use a helpful function from `sklearn.model_selection` called `train_test_split`. You can have a look at the documentation: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html. Note that `train_test_split` defaults to randomly sampling the data to split it into training and

validation/test sets, that is, the default value is `shuffle=True`

For reproducibility, we first fix the value in the numpy random seed about the state of randomness. This ensures that, every step including randomness, will produce the same output if we re-run the code or if someone else wants to reproduce our results (e.g. produce the same train-test split).

In `sklearn`, the suggestion to control randomness across multiple consecutive executions is as follows:

- In order to obtain reproducible (i.e. constant) results across multiple program executions, we need to remove all uses of `random_state=None`, which is the default.
- Declare your own `rng` variable (random number generator) at the top of the program, and pass it down to any object that accepts a `random_state` parameter. You can check some details from [here](https://numpy.org/doc/1.16/reference/generated/numpy.random.RandomState.html); <https://numpy.org/doc/1.16/reference/generated/numpy.random.RandomState.html>

Thus, our first step before splitting the data is to define our `rng` variable.

```
[8]: # To make this notebook's output identical at every run
rng = np.random.seed(0)
# might be good for our course
# np.random.seed(11205)
```

3.1.1 Exercise 2 (CORE)

Run the following code to use `train_test_split()` to split the data randomly into training (70%) and test (30%) sets. The `training set` will contain our training data, called `X_train` and `y_train`. The `test set` will contain our testing data, called `X_test` and `y_test`.

Can you think on a scenario where not shuffling would be a good idea? What about when we do want to shuffle our data?

```
[9]: # As a good practice first split the data
from sklearn.model_selection import train_test_split

X = df_insurance.drop('charges', axis = 1) # Set of features
y = df_insurance['charges']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.3,
                                                    random_state = rng)
```

- In lots of real world ML applications you train on historical data, and make predictions on future unseen data. If this was the case we may want our data sorted by timestamp before creating the splits, since it matches the way you'll apply the model in the real world.
- However, in this case where we do not have a timestamp, we probably want to shuffle our data first because the ordering may effect our assessment of generalisation performance.
- The ordering of the data depends on where it came from and how it was exported and it's not uncommon that real world data is sorted in some manner.

- Therefore if there is an order according to one of the attributes, we could have a very poor test set.

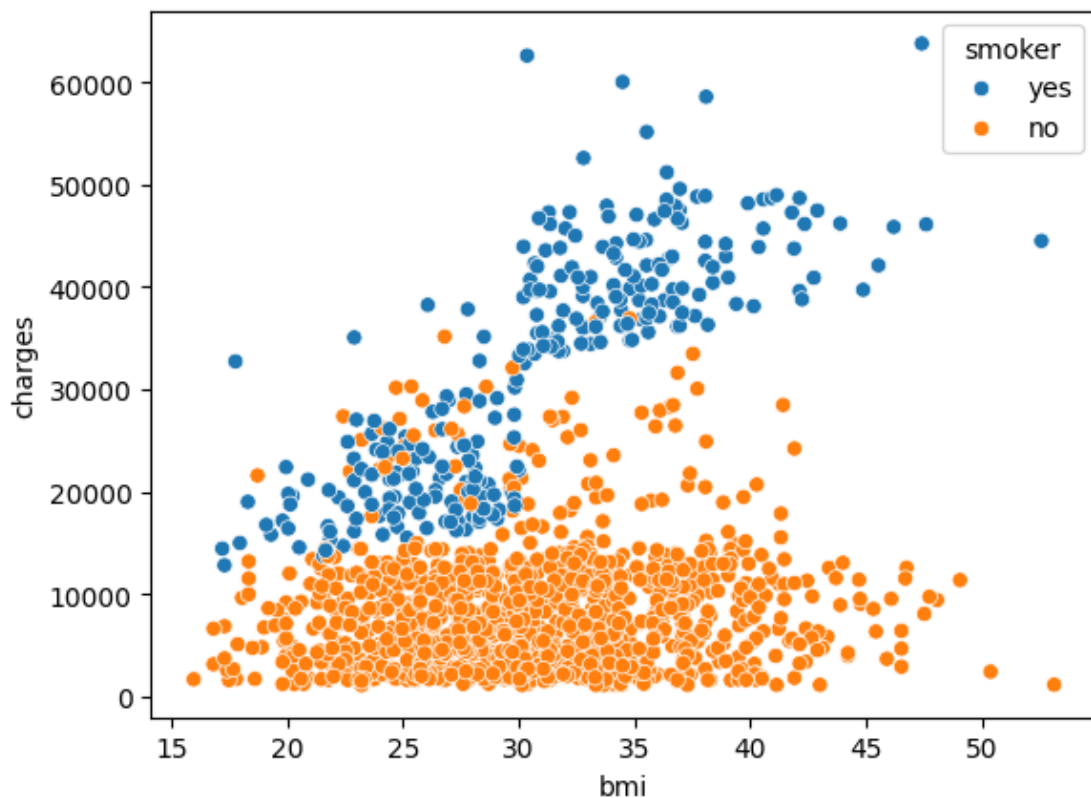
4 Least Squares Estimation

Consider a linear regression model for `charges` using `bmi` and `smoker` as features in our model. Without sklearn functionalities, let's compute and visualize the least squares estimates.

4.0.1 Exercise 3 (CORE)

Create a scatter plot using `sns.scatterplot` of `charges` vs `bmi`, colored by `smoker`. Describe any apparent relationship between `charges` and `bmi` and comment on the difference between smokers and non-smokers.

```
[10]: # Scatter of charges v bmi, colored by smoker
sns.scatterplot(x="bmi", y="charges", hue="smoker", data = df_insurance)
plt.show()
```



- There does appear to be a relationship between `bmi` and `charges`. A linear relationship may be a good baseline model to start with.
- There is an apparent difference in the smokers and non-smokers, and scatterplot suggest that different slopes may also be needed within each class.

4.0.2 Exercise 4 (CORE)

Now, let's compute the least square estimates.

First let's construct the feature matrix as an `np.array`. Recall that we need to include a column of ones to allow a non-zero intercept. We also need to convert the response `y_train` to an `np.array`.

```
[11]: # Writing the design matrix
X = np.c_[
    np.ones(len(X_train)),
    X_train.bmi,
    X_train.smoker == "yes"
]

# Converting response training object to numpy array
y = np.array(y_train)

X.shape, y.shape
```

```
[11]: ((936, 3), (936,))
```

- Next, compute the least squares estimates $\hat{w}$, using the expression from lectures and the `solve` function from `numpy.linalg`.
- What is the intercept for non-smokers and what is the intercept for smokers?

```
[12]: # Part a
from numpy.linalg import solve

w = solve(X.T @ X, X.T @ y)

print(w)

[-3385.172326      389.05002587 22926.48168453]
```

```
[13]: # Part b

print("The intercept for non-smokers is: %.3f" % w[0])

print("The intercept for smokers is: %.3f" % (w[0]+w[2]))
```

The intercept for non-smokers is: -3385.172

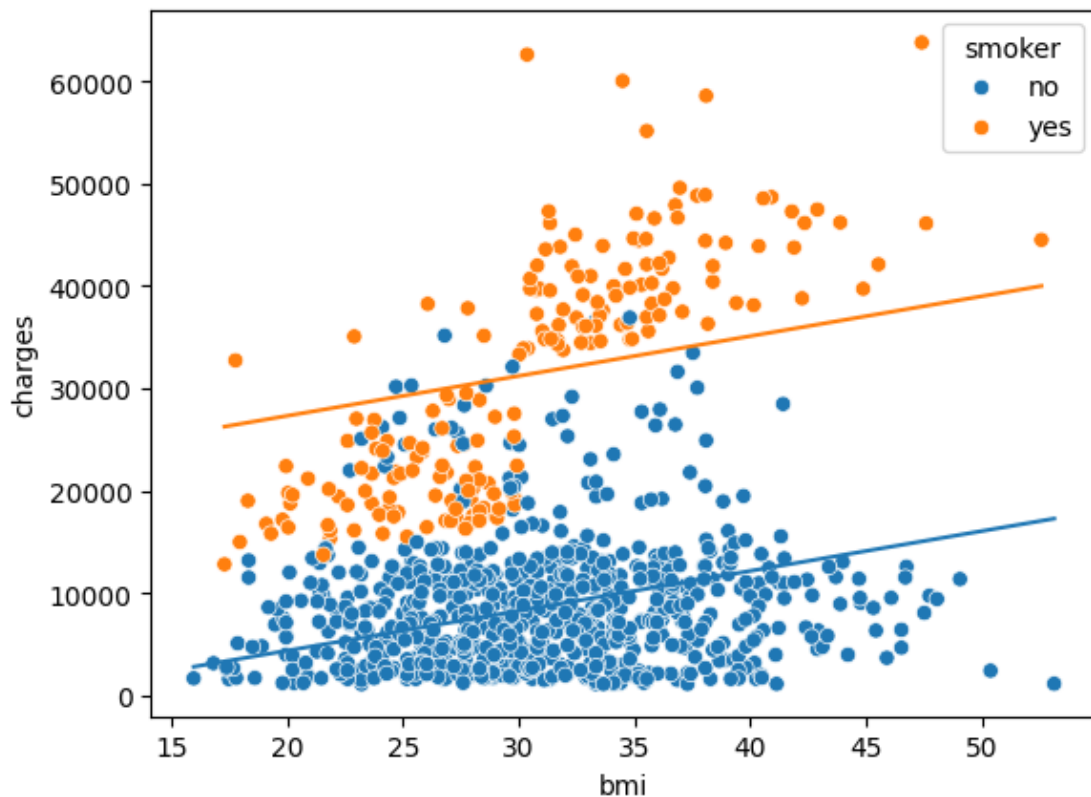
The intercept for smokers is: 19541.309

4.0.3 Exercise 5 (CORE)

- Compute the fitted values from this model by calculating $\hat{y} = X\hat{w}$.
- Redraw your scatter plot of `charges` vs `bmi`, colored by `smoker` from Exercise 3, and overlay a line plot of the fitted values (fitted regression line) using `sns.lineplot`. Comment on the results and any potential feature engineering steps that could help to improve the model.

```
[14]: # Part a: Compute fitted values
yhat = X @ w

# Part b: Plot fitted values
ax = sns.scatterplot(x = X_train.bmi, y = y, hue = X_train.smoker)
sns.lineplot(x = X_train.bmi, y = yhat, hue = X_train.smoker, ax=ax, legend =_
↪False)
ax.set(ylabel='charges')
plt.show()
```



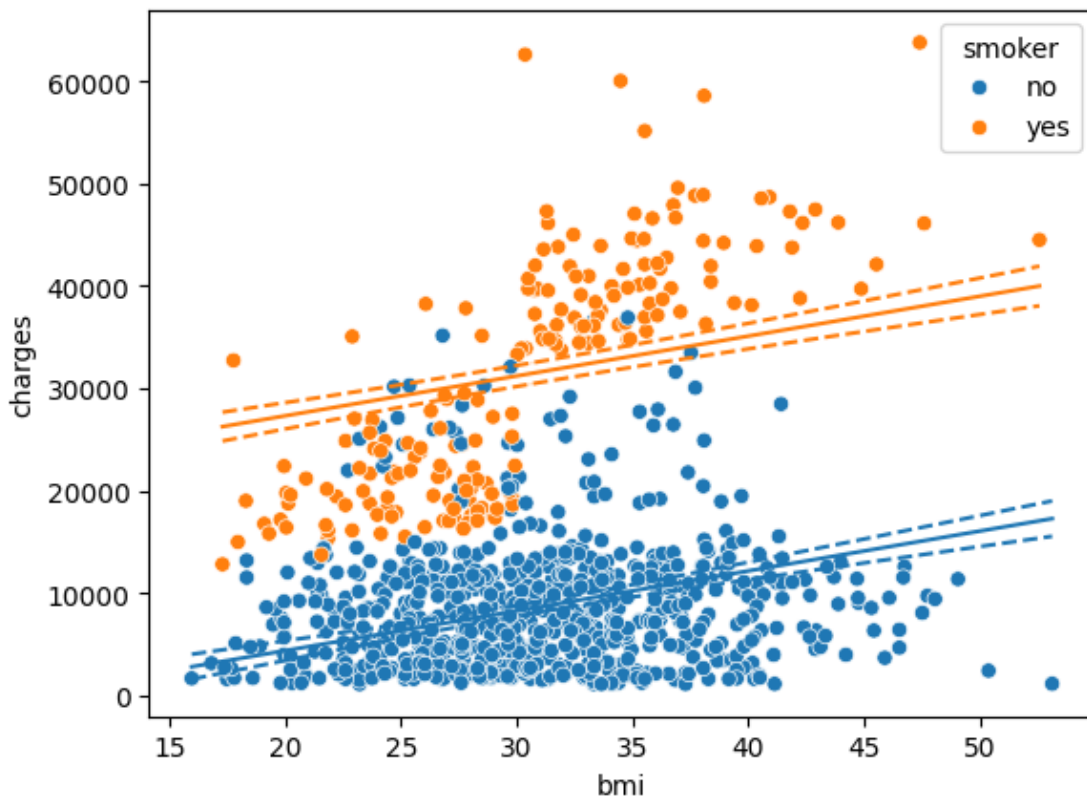
There is an evident increase in predicted medical charges for smokers and an increase with higher bmi. This is a sensible baseline model, but it could be improved, for example by including an interaction between smoker and bmi (visually, it appears the slope may be steep for smokers) and possibly also nonlinear terms.

4.0.4 Exercise 6 (EXTRA)

In the following code following code, we compute the standard error for the predictions following the expression in lectures. Add dashed lines to your plot representing the prediction intervals for smokers and non-smokers.

```
[15]: # Compute standard error of predictions
sigmsq = np.sum((y - yhat)**2) / (X.shape[0] - X.shape[1]-1)
sd_err_pred = np.array([np.sqrt(sigmsq * X[i,:] @ np.linalg.solve(X.T@X, X[i,:
↪].T)) for i in range(X.shape[0])])

[16]: # Plot fitted values with prediction intervals
ax = sns.scatterplot(x = X_train.bmi, y = y, hue = X_train.smoker)
sns.lineplot(x = X_train.bmi, y = yhat, hue = X_train.smoker, ax=ax, legend =_
↪False)
sns.lineplot(x = X_train.bmi, y = yhat + 1.96 * sd_err_pred,
             linestyle='--', hue = X_train.smoker, ax=ax, legend = False)
sns.lineplot(x = X_train.bmi, y = yhat - 1.96 * sd_err_pred,
             linestyle='--', hue = X_train.smoker, ax=ax, legend = False)
ax.set(ylabel='charges')
plt.show()
```



4.1 Residuals

A useful tool for evaluating a model is to examine the residuals of that model. For any standard regression model, the residual for observation n is defined as $y_n - \hat{y}_n$ where $\hat{y}_n$ is the model's fitted value for observation n .

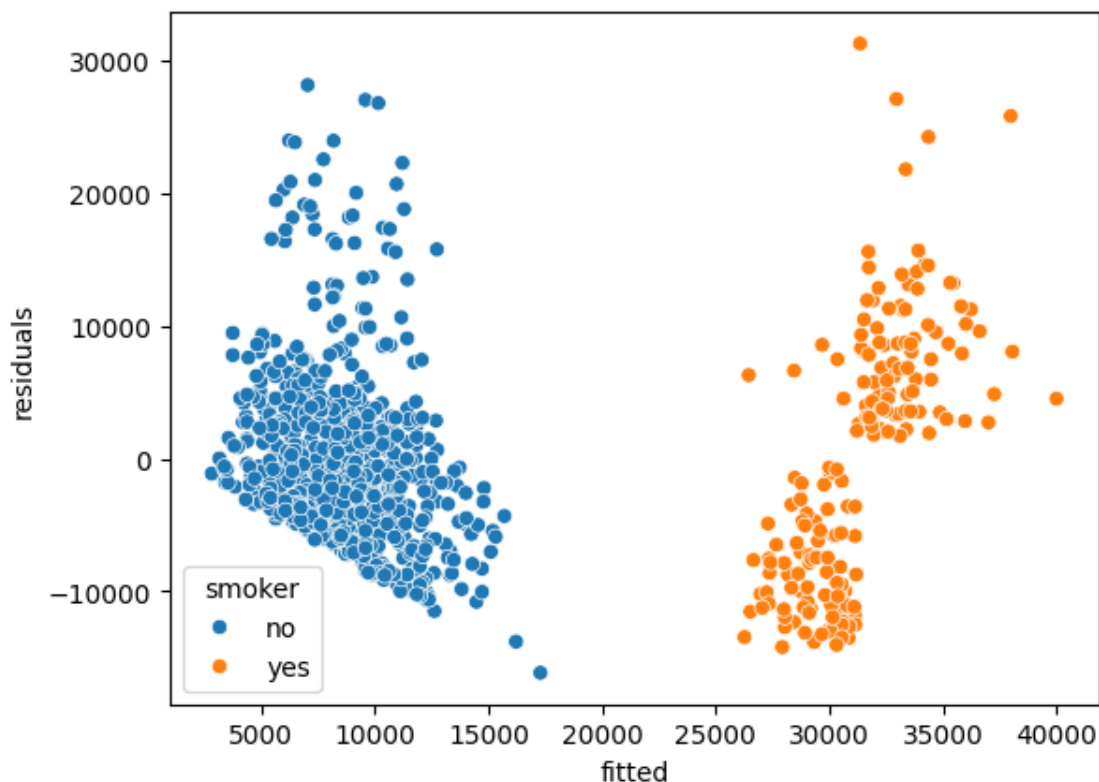
Studying the properties of the residuals is important for assessing the quality of the fitted regression model. This scatterplot (fitted vs residuals) gives us more intuition about the model performance. Briefly,

- If the normal linear model assumption is true then the residuals should be randomly scattered around zero with no discernible clustering or pattern with respect to the fitted values.
- Furthermore, this plot can be useful to check the constant variance (homoscedastic) assumption to see whether the range of the scatter of points is consistent over the range of fitted values.

4.1.1 Exercise 7 (CORE)

- a) Calculate the residuals and create a residual plot (scatter plot of fitted vs residuals) for this model and color by smoker. Comment on quality of the model based on this plot.
- b) Compute the R^2 value for this model and comment on its value (recall from lectures that R^2 is 1 minus the sum of the squared residuals divided by the sum of squared differences between y and its mean).

```
[17]: # Part a
resid = y - yhat
ax = sns.scatterplot(x = yhat, y = resid, hue = X_train.smoker)
ax.set(xlabel = 'fitted', ylabel='residuals')
plt.show()
```



Within each group, the points do not seem to be randomly scattered around zero and there appears to be a pattern. In particular for non-smokers, the residuals tend to be greater than zero for smaller fitted values (underestimation) and the residuals tend to be smaller than zero for larger fitted values (overestimation). The converse appears to be true for smokers. Since the model assumes a common slope for both smokers and non-smokers, the behavior across the two groups is averaged. Allowing different slopes for smokers and non-smokers may help address this.

```
[18]: # Part b
R_2 = 1 - sum(resid**2)/sum((y - np.mean(y))**2)
print(R_2)
```

0.6350967450118408

The R^2 value is fairly good. The model explains about 63.5% of the variability of the response variable around its mean.

5 Regression using scikit-Learn

Linear regression is available in **scikit-learn** (**sklearn**) through `LinearRegression` from the `linear_model` submodule. You can browse through the documentation and examples [here](#). Let's start by importing it.

```
[19]: from sklearn.linear_model import LinearRegression
```

In general sklearn's models are implemented by first creating a model object, and then using that object to fit your data. As such, we will now create a linear regression model object `lr` and use it to fit our data. Once this object is created we use the `fit` method to obtain a model object fitted to our data.

Note: by default an intercept is included in the model. So, we do NOT need to add a column of ones to our design matrix.

Note: in the code below we use `make_column_transformer` which automatically names all of our transformers. To see the names, run `preprocessor.named_transformers_`. You can then access each transformer by its name, if you want to view its attributes or call methods on it, e.g. `preprocessor['onehotencoder'].get_feature_names_out()`.

Note: using the option `OneHotEncoder(drop=np.array(['Reference Category']))`, we can specify the which category to drop (the reference category) by replacing 'Reference Category' with the desired category.

```
[20]: lr = LinearRegression()

# Create feature matrix
from sklearn.compose import make_column_transformer
from sklearn.preprocessing import OneHotEncoder
```

```

from sklearn.preprocessing import StandardScaler
preprocessor = make_column_transformer(
    (OneHotEncoder(drop=np.array(['no'])), ['smoker']), # encode 'smoker' column
    (StandardScaler(), ['bmi']), # scale 'bmi' column
    remainder='drop', # drop everything else
    verbose_feature_names_out=False)
X_train_ = preprocessor.fit_transform(X_train)

lr_fit = lr.fit(
    X = X_train_,
    y = y_train
)

# To see the names of the transformers
# preprocessor.named_transformers_
# You can call a method or access attributes of each transformer by its name
# preprocessor['onehotencoder'].get_feature_names_out()

```

This model object then has various useful methods and attributes, including `intercept_` and `coef_` which contain our estimates for w .

Note: if `fit_intercept=False` and a column of ones is included in the design matrix, then both the intercept and coefficient will be stored in `coef_`.

```

[21]: w_0 = lr_fit.intercept_ # Intercept term of the fitted model
      w_1 = lr_fit.coef_

      w = np.concatenate([w_0, w_1])
      feature_names = ['intercept'] + list(preprocessor.get_feature_names_out())
      w_df = pd.DataFrame({'feature': feature_names, 'coefficient': w})
      w_df

```

```

[21]:      feature  coefficient
0  intercept   8579.036638
1  smoker_yes  22926.481685
2         bmi   2424.659184

```

Importantly, to avoid *data leakage* and improve *reproducibility*, we should always be combining our feature engineering and model fitting steps into a single **pipeline**.

Note: in the code below we use `make_pipeline` which automatically names all of our transformers/estimators. To see the names, run `preprocessor.named_steps`. You can then access each step by its name or its index in the pipeline, e.g. `pipe['columntransformer'].get_feature_names_out()`.

```

[22]: # Combine into pipeline
      from sklearn.pipeline import make_pipeline
      pipe = make_pipeline(
          preprocessor,

```



```

    LinearRegression()
)

pipe_fit = pipe.fit(
    X = X_train,
    y = y_train
)

# You can access the coefficients by the index of the final step
pipe[-1].coef_
# Or by its name
# pipe.named_steps
# pipe['linearregression'].coef_

```

[22]: array([22926.48168453, 2424.65918409])

The model fit objects also provides additional useful methods for evaluating the model R^2 (**score**) and calculating predictions (**predict**). Let's use the latter to compute the fitted values and predictions, as well as some metrics to evaluate the performance on the test data.

```

[23]: # Fitted values
y_fit = pipe.predict(X_train)

# Predicted values
y_pred = pipe.predict(X_test)

# The mean squared error of the training set
print("Training Mean squared error: %.3f" % mean_squared_error(y_train, y_fit))
# The R squared of the training set
print("Training R squared: %.3f" % r2_score(y_train, y_fit))

# The mean squared error of the test set
print("Test Mean squared error: %.3f" % mean_squared_error(y_test, y_pred))
# The R squared of the test set
print("Test R squared: %.3f" % r2_score(y_test, y_pred))

# Another way for R2 calculation
print(pipe.score(X_train, y_train))

```

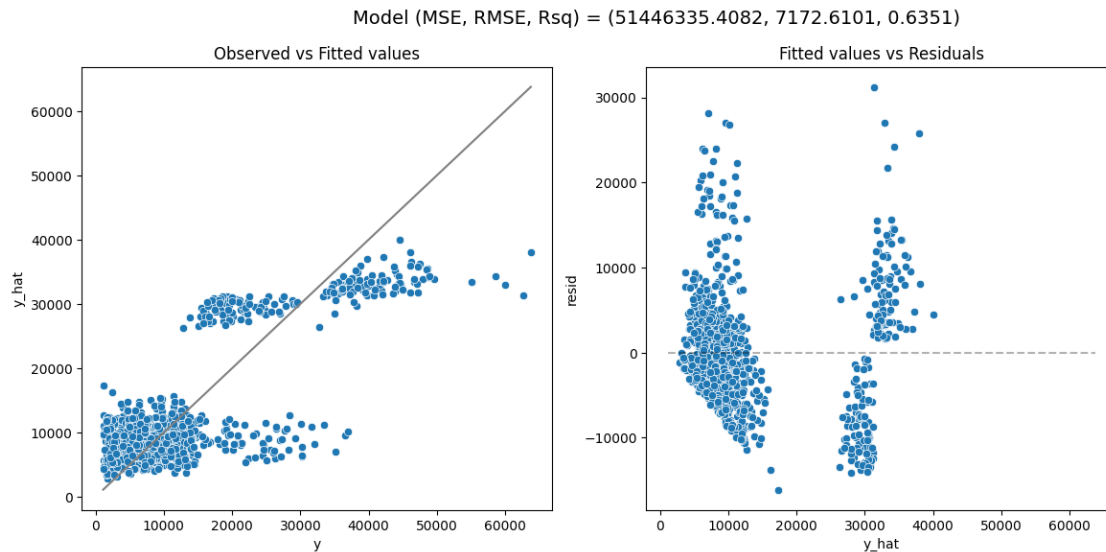
```

Training Mean squared error: 51446335.408
Training R squared: 0.635
Test Mean squared error: 47297000.691
Test R squared: 0.703
0.6350967450118401

```

Or we can use our helper function and also plot the fitted values and residuals to identify if there are potential patterns.

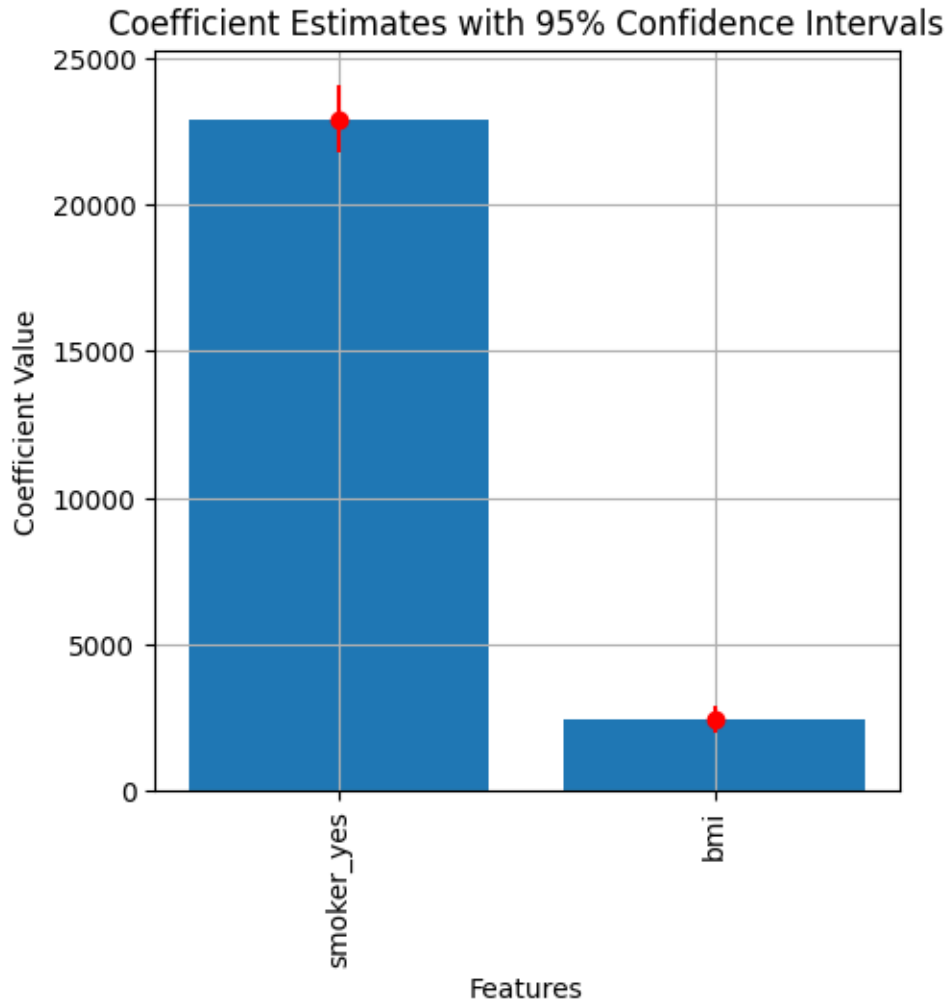
```
[24]: # If we use the pre-defined function
model_fit(pipe, X_train, y_train, plot = True)
```



```
[24]: (51446335.408195145, 7172.610083379352, 0.6350967450118401)
```

Lastly, we can use our helper function to compute standard errors of the coefficients and plot them.

```
[25]: sd, cov_mat = compute_std_error(pipe, X_train, y_train, plot=True,
    ↪ feature_names=['smoker_yes', 'bmi'])
```



5.0.1 Exercise 8 (CORE)

Next, modify your pipeline to also include `age` as feature.

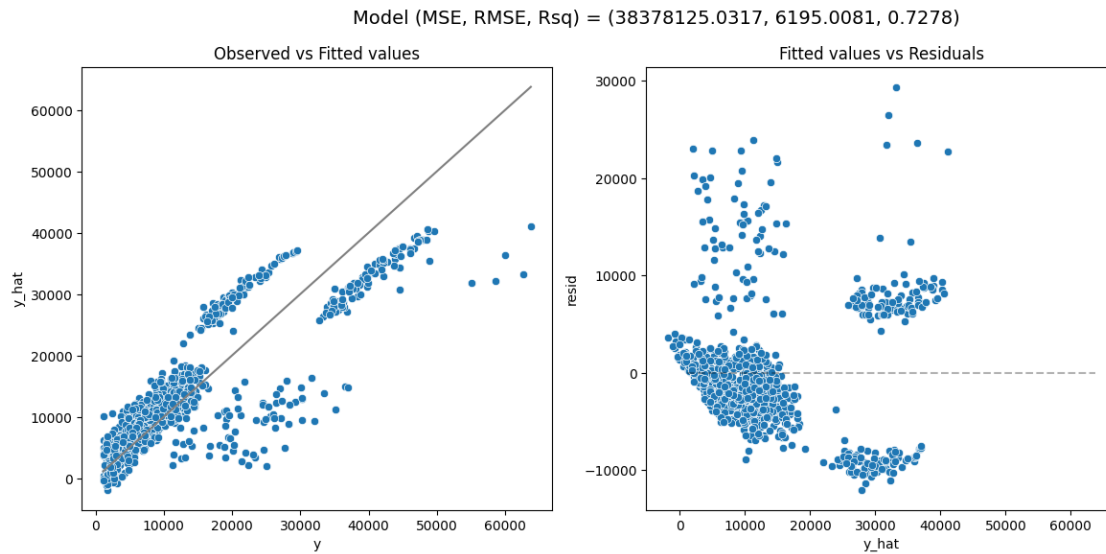
Fit the model on the training data and calculate performance metrics similar to above (R^2 , and MSE / RMSE). How does this model compare to previous one?

```
[26]: # Modify preprocessor to include 'age' as a feature
preprocessor = make_column_transformer(
    (OneHotEncoder(drop=np.array(['no'])), ['smoker']), # encode 'smoker' column
    (StandardScaler(), ['bmi', 'age']), # scale 'bmi' and 'age' columns
    remainder='drop', # drop everything else
    verbose_feature_names_out=False)

# Create and fit new pipeline
pipe2 = make_pipeline(
```

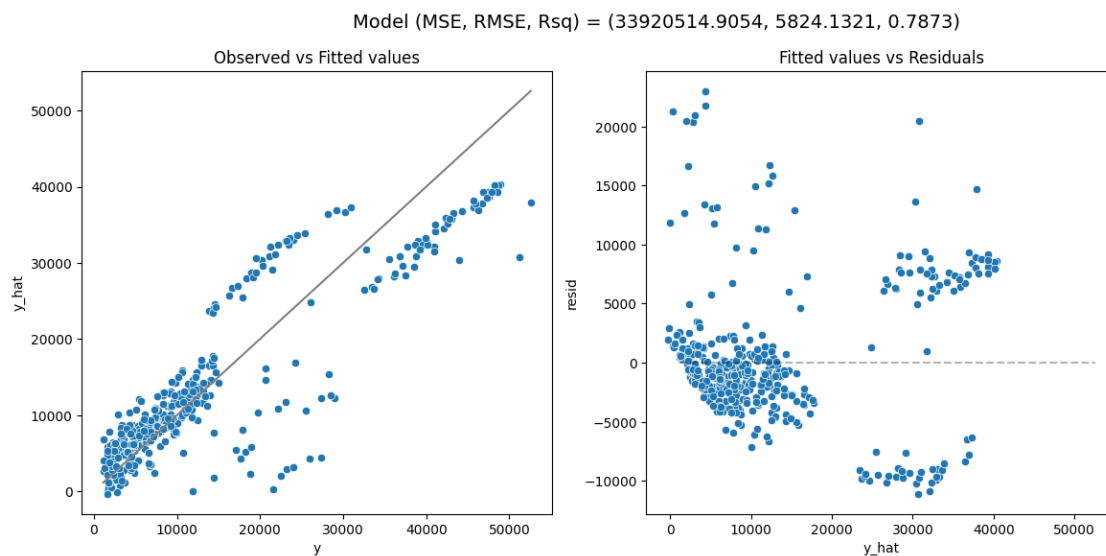
```
preprocessor,
  LinearRegression()
).fit(X_train, y_train)
```

```
[27]: # Using the model_fit function on training data
model_fit(pipe2, X_train, y_train, plot = True)
```



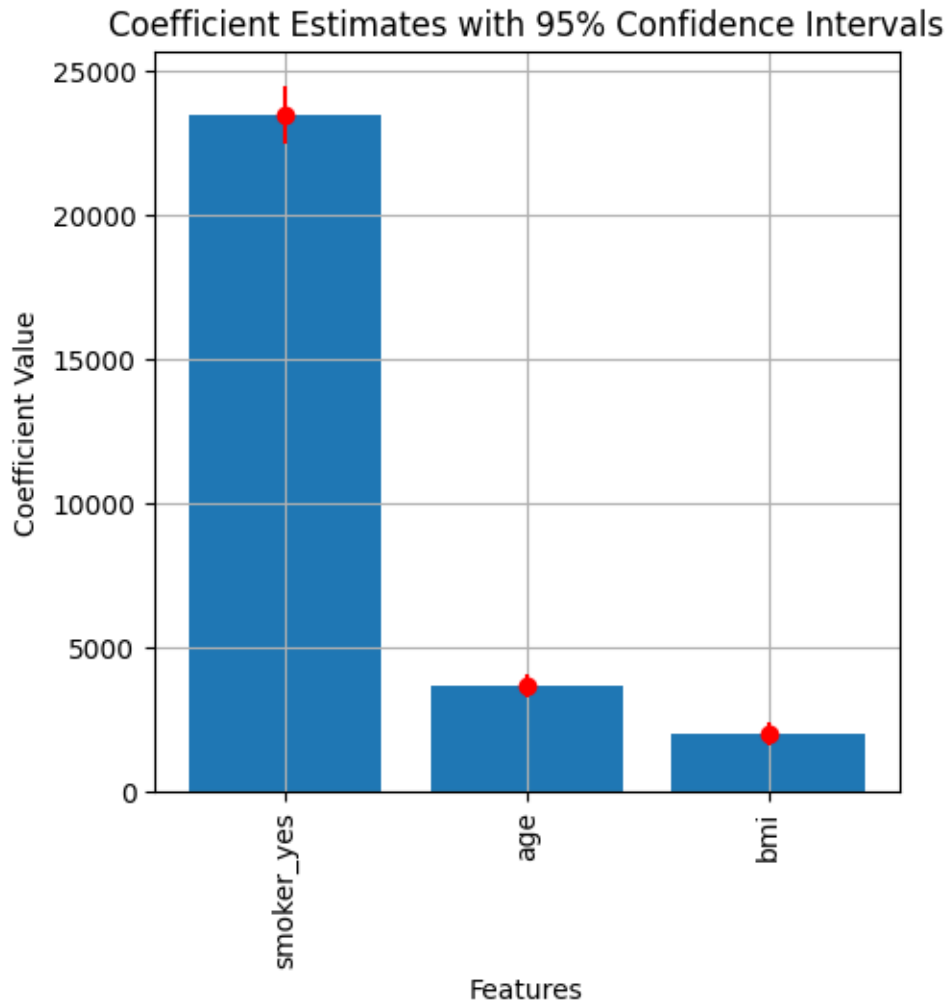
```
[27]: (38378125.03170811, 6195.008073578929, 0.7277881381968745)
```

```
[28]: # Using the model_fit function on test data
model_fit(pipe2, X_test, y_test, plot = True)
```



[28]: (33920514.90536016, 5824.132116063316, 0.7872914737498631)

```
[29]: sd, cov_mat = compute_std_error(pipe2, X_train, y_train, plot=True,
    ↪ feature_names=['smoker_yes', 'bmi', 'age'])
```



Including age helps to improve the model, providing better performance metrics. As we have included an additional feature **age**, the training MSE/RMSE is lower (higher R²) as expected, but the test MSE/RMSE is also lower (higher R²).

We do still notice some patterns in the residuals. For example, the evident under and over estimation for smokers with low and high bmi.

6 Polynomial Regression

6.1 Polynomial features in sklearn

sklearn has a built in function called `PolynomialFeatures` which can be used to simplify the process of including polynomial features in a model. You can browse the documentation [here](#). This function is included in the *preprocessing* module of sklearn, as with other python functions we can import it as follows.

```
[30]: from sklearn.preprocessing import PolynomialFeatures
```

Construction and use of this is similar to what we have already seen with other transformers; we construct a `PolynomialFeatures` object in which we set basic options (e.g. the degree of the polynomial) and then apply the transformation to our data by calling `fit_transform`. This will generate a new model matrix which includes the polynomial features up to the degree we have specified.

Run the following code for a simple illustration:

```
[31]: x = np.array([1, 2, 3, 4])
      PolynomialFeatures(degree = 2).fit_transform(x.reshape(-1,1))
```

```
[31]: array([[ 1.,  1.,  1.],
             [ 1.,  2.,  4.],
             [ 1.,  3.,  9.],
             [ 1.,  4., 16.]])
```

Note that when we use this transformation, we get **all of the polynomial transformations of x from 0 to degree**.

In this case, the **0 degree column** is equivalent to **the intercept column**. If we do not want to include this we can construct `PolynomialFeatures` with the option `include_bias=False`.

```
[32]: PolynomialFeatures(degree = 2, include_bias=False).fit_transform(x.
      ↪ reshape(-1,1))
```

```
[32]: array([[ 1.,  1.],
             [ 2.,  4.],
             [ 3.,  9.],
             [ 4., 16.]])
```

We can also use `PolynomialFeatures` to add only interaction terms, through the option `interaction_only=True`. As an illustration run the following code:

```
[33]: x = np.array([[1, 2, 3, 4],[0,0,1,1]]).T
      PolynomialFeatures(interaction_only=True,include_bias=False).fit_transform(x)
```

```
[33]: array([[1., 0., 0.],
             [2., 0., 0.],
             [3., 1., 3.],
             [4., 1., 4.]])
```

6.2 Interactions

Now, let's create a pipeline that:

- includes `bmi` as a numerical variable and `smoker` condition as a categorical variable
- applies encoding for the `smoker` variable
- uses `PolynomialFeatures` to include an **interaction** between `smoker` and `bmi`

```
[34]: # Create Pipeline for model that includes interactions
preprocessor = make_column_transformer(
    (OneHotEncoder(drop=np.array(['no'])), ['smoker']), # encode 'smoker' column
    (StandardScaler(), ['bmi']), # scale 'bmi' column
    verbose_feature_names_out=False)

# Create and fit new pipeline
pipe3 = make_pipeline(
    preprocessor,
    PolynomialFeatures(interaction_only=True, include_bias=False), # add extra
    ↪step for interaction terms
    LinearRegression()
).fit(X_train, y_train)
```

Note: we set `include_bias=False` as the intercept is included in linear regression by default.

We can also extract the **names of the features** using the method `get_feature_names_out()`.

```
[35]: # Names after the first transformer
print(pipe3[0].get_feature_names_out())

# Names after the first transformer and interaction step
print(pipe3[:-1].get_feature_names_out())

# Create a data frame to see the intercept and coefficients
w = [pipe3[-1].intercept_] + pipe3[-1].coef_.tolist()
feature_names = ['intercept'] + list(pipe3[:-1].get_feature_names_out())
w_df = pd.DataFrame({'feature': feature_names, 'coefficient': w})
w_df
```

```
['smoker_yes' 'bmi']
['smoker_yes' 'bmi' 'smoker_yes bmi']
```

```
[35]:
```

	feature	coefficient
0	intercept	8606.815570
1	smoker_yes	23291.340426
2	bmi	637.827154
3	smoker_yes bmi	8219.259903

6.2.1 Exercise 9 (CORE)

For the model trained above (`pipe3`):

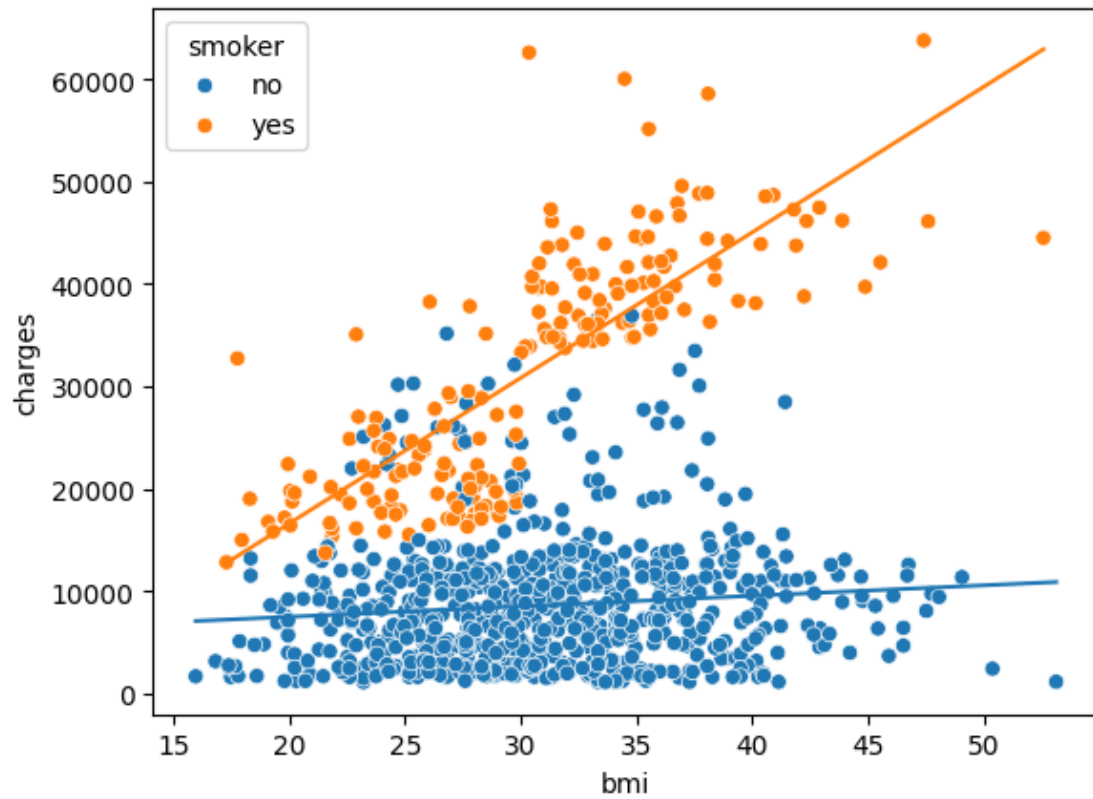
- a) What is the intercept and slope for non-smokers and what is the intercept and slope for smokers?
- b) Compute the fitted values by calling `predict`. Draw the scatter plot of bmi against charges, colored by smoker (from Exercise 3), and overlay a line plot of the fitted values (fitted regression lines).
- c) How does this model compare to our previous models `pipe` and `pipe2`?

```
[36]: #Part a
print("The intercept for non-smokers is: %.3f" % w[0])
print("The slope for non-smokers is: %.3f" % w[2])
print("The intercept for smokers is: %.3f" % (w[0]+w[1]))
print("The slope for smokers is: %.3f" % (w[2]+w[3]))
```

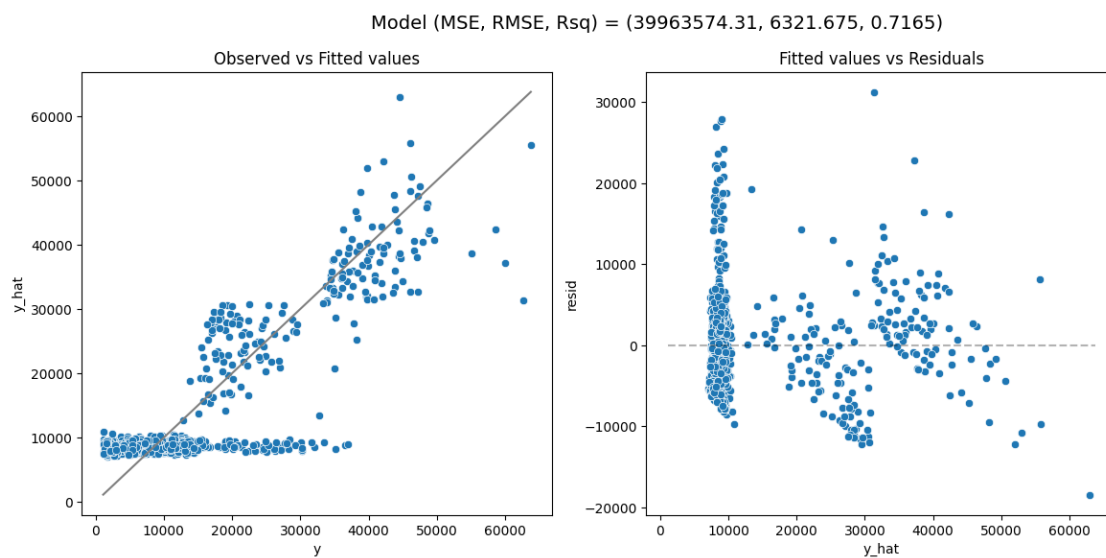
```
The intercept for non-smokers is: 8606.816
The slope for non-smokers is: 637.827
The intercept for smokers is: 31898.156
The slope for smokers is: 8857.087
```

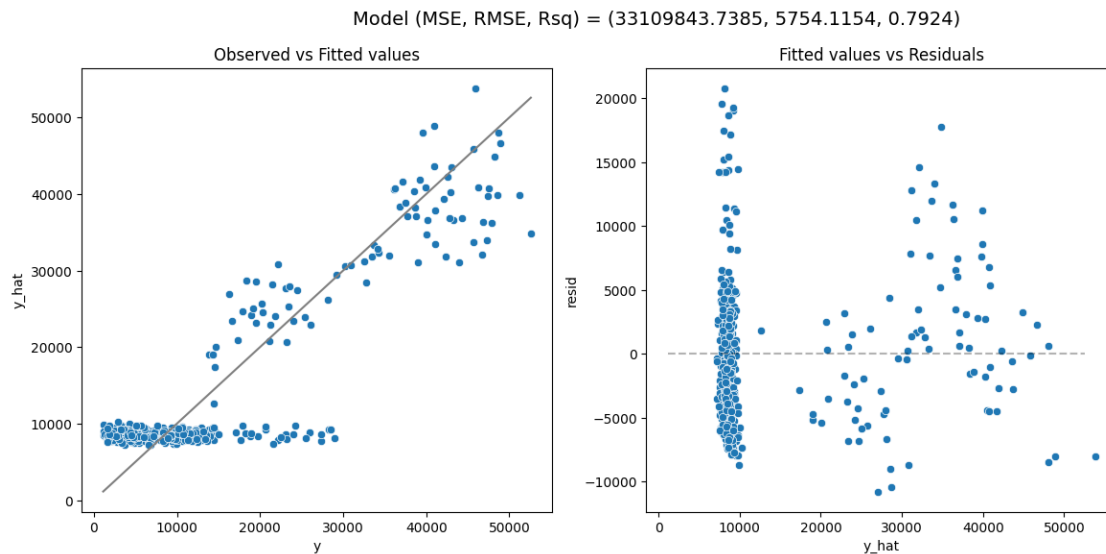
```
[37]: #Part b
# Compute fitted values
yhat = pipe3.predict(X_train)

# Part b: Plot fitted values
ax = sns.scatterplot(x = X_train.bmi, y = y_train, hue = X_train.smoker)
sns.lineplot(x = X_train.bmi, y = yhat, hue = X_train.smoker, ax=ax, legend =
↪False)
ax.set(ylabel='charges')
plt.show()
```

```
[38]: # Part c
model_fit(pipe3, X_train, y_train, plot = True)
model_fit(pipe3, X_test, y_test, plot = True)
```





[38]: (33109843.738532394, 5754.115374106814, 0.7923750248000321)

Including the interaction term, results in improved performance metrics and visually seems to fit the data better, especially compared to `pipe`. We do still observe some patterns in the residuals, such as higher variance in the residuals for non-smokers, but there is less structure than previously. We may also want to include `age` as additional input to further improve the model.

6.3 Nonlinearity

Now, let's explore including nonlinearity into the model through a polynomial basis function expansions of `bmi`.

6.3.1 Exercise 10 (CORE)

First, suppose you naively apply a polynomial basis function expansion to the feature matrix containing `bmi` and `smoker`:

- Run the code below to create the transformed feature matrix using `PolynomialFeatures` assuming `degree=2`.
- Why should we **NOT** use this naive polynomial basis function expansion?

```
[39]: # Create a transformed feature matrix containing a polynomial expansion (degree=2) of bmi and smoker

# Create Pipeline for model that includes interactions
preprocessor = make_column_transformer(
    (OneHotEncoder(drop=np.array(['no'])), ['smoker']), # encode 'smoker' column
    (StandardScaler(), ['bmi']), # pass through 'bmi' column
```

```

        verbose_feature_names_out=False)

# Create and fit new pipeline
fe = make_pipeline(
    preprocessor,
    PolynomialFeatures(degree=2), # add step for polynomial features
)

Xt = fe.fit_transform(X_train)
Xt_ = pd.DataFrame(Xt, columns=fe.get_feature_names_out())
Xt_.head()

```

```

[39]:      1  smoker_yes      bmi  smoker_yes^2  smoker_yes bmi      bmi^2
0  1.0          0.0 -0.407135          0.0        -0.0  0.165759
1  1.0          0.0  0.328554          0.0          0.0  0.107948
2  1.0          0.0  2.566909          0.0          0.0  6.589022
3  1.0          0.0 -0.696757          0.0        -0.0  0.485470
4  1.0          0.0 -0.529081          0.0        -0.0  0.279926

```

Since `smoker` is binary, the third and sixth columns/features are the same. We do **NOT** want to apply basis expansion to the binary features, as this creates a rank deficient matrix and redundant features.

6.3.2 Exercise 11 (CORE)

Now, let's create a model that allows nonlinearity of `bmi` through polynomial basis function expansion of degree 3 (with no interactions for ease of exposition).

Create a new pipeline to construct this model. Train this model and then plot the fitted regression line and compute the performance metrics.

```

[40]: # Create Pipeline for model that includes polynomial expansions of bmi
num_pre = make_pipeline(
    StandardScaler(),
    PolynomialFeatures(degree=3, include_bias=False)
)

preprocessor = make_column_transformer(
    (OneHotEncoder(drop=np.array(['no'])), ['smoker']), # encode 'smoker' column
    (num_pre, ['bmi']), # polynomial features of 'bmi' column
    verbose_feature_names_out=False)

# Create the new pipeline
polyreg_pipe = make_pipeline(
    preprocessor,
    LinearRegression(), # add step for polynomial features
)

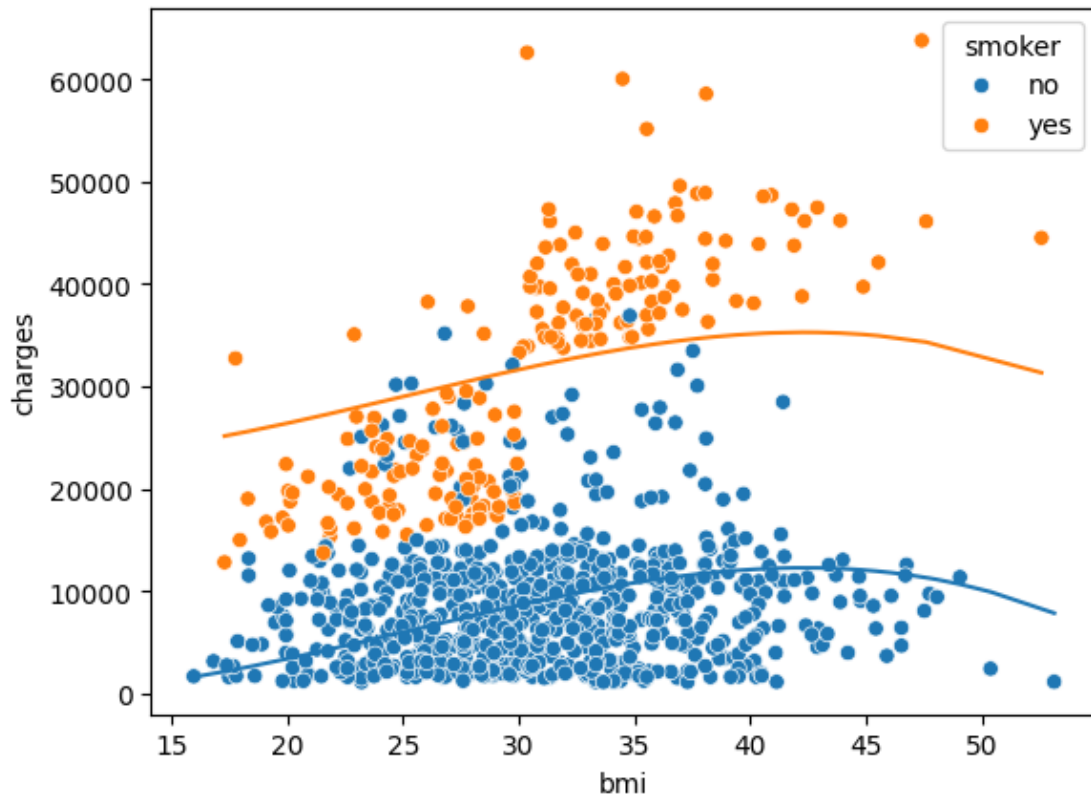
#Train the model

```

```
polyreg_pipe.fit(X_train,y_train)
```

```
[40]: Pipeline(steps=[('columntransformer',  
                        ColumnTransformer(transformers=[('onehotencoder',  
OneHotEncoder(drop=array(['no'], dtype='<U2')),  
                                                         ['smoker'])),  
                        ('pipeline',  
Pipeline(steps=[('standardscaler',  
StandardScaler()),  
                  ('polynomialfeatures',  
PolynomialFeatures(degree=3,  
                    include_bias=False))])),  
                                                         ['bmi'])]),  
              verbose_feature_names_out=False)),  
        ('linearregression', LinearRegression())])
```

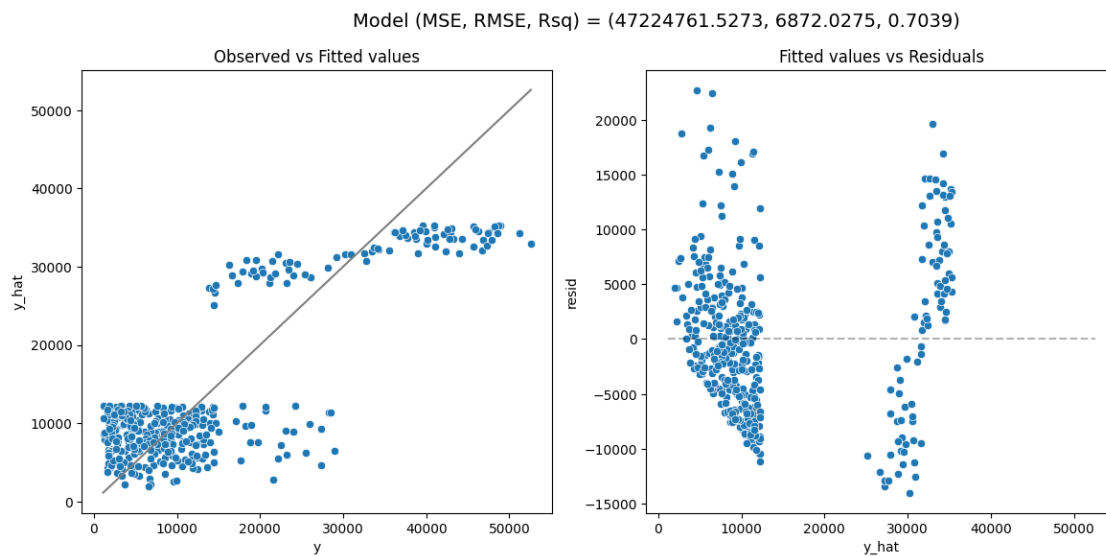
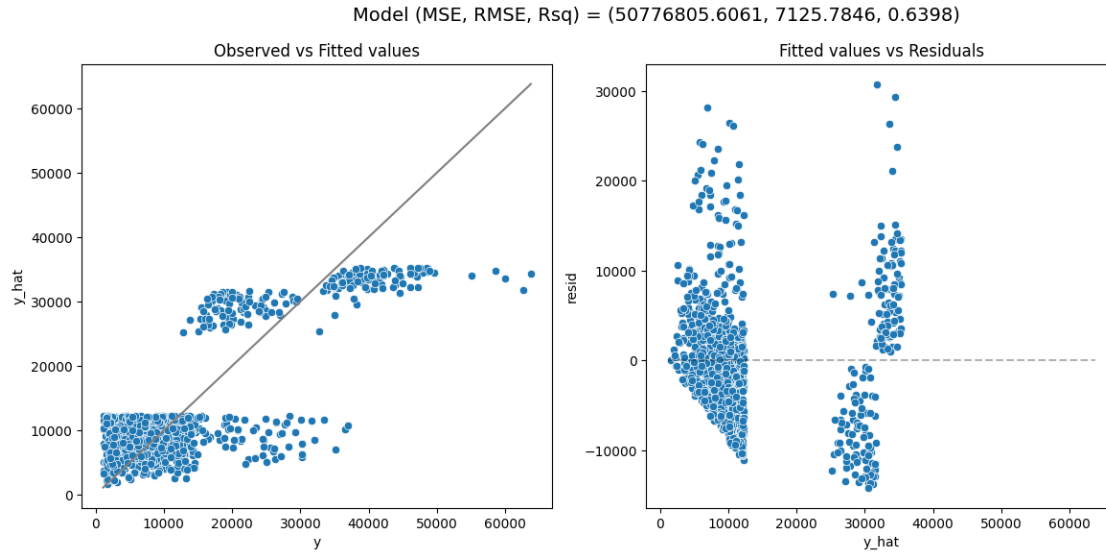
```
[41]: # Compute fitted values  
yhat = polyreg_pipe.predict(X_train)  
  
# Plot fitted values  
ax = sns.scatterplot(x = X_train.bmi, y = y_train, hue = X_train.smoker)  
sns.lineplot(x = X_train.bmi, y = yhat, hue = X_train.smoker, ax=ax, legend =  
↳False)  
ax.set(ylabel='charges')  
plt.show()
```



```
[42]: # Create a data frame to see the intercept and coefficients
w = [polyreg_pipe[-1].intercept_] + polyreg_pipe[-1].coef_.tolist()
feature_names = ['intercept'] + list(polyreg_pipe[:-1].get_feature_names_out())
w_df = pd.DataFrame({'feature': feature_names, 'coefficient': w})
w_df
```

```
[42]:      feature  coefficient
0  intercept    9010.405147
1  smoker_yes  22996.565166
2      bmi      3008.026545
3    bmi^2     -395.364310
4    bmi^3     -147.969291
```

```
[43]: model_fit(polyreg_pipe, X_train, y_train, plot = True)
model_fit(polyreg_pipe, X_test, y_test, plot = True)
```



[43]: (47224761.52728303, 6872.027468461039, 0.7038632976236102)

6.4 Choosing the Order of the Polynomial

How can we choose the order of the polynomial?

In lecture, we discussed how choosing the degree to be too large can cause over fitting. When we over fit a polynomial regression model, the MSE for the training data will appear to be low which might indicate that the model is a good fit. But, as a result of over fitting, the MSE for the predictions of the unseen test data may begin to increase. However, we can **NOT** use the test to

determine the order of the polynomial, so in the following, we explore using cross-validation to do so.

6.4.1 Tuning with GridSearchCV

If we wish to test over a specific set of parameter values using cross validation we can use the `GridSearchCV` function from the `model_selection` submodule. In this setting, the hyperparameter is actually the degree of the polynomial that we are investigating.

This argument is a dictionary containing parameters names as keys and lists of parameter settings to try as values. Since we are using a pipeline, our parameter name will be the name of the pipeline step, `columntransformer`, followed by `__`, (then, the name of next step if applicable, e.g. `pipeline__` since we are using `Pipeline` for the steps of the numerical features, and `polynomialfeatures__` for the polynomial expansion), and then the parameter name, `degree`. So for our pipeline the parameter is named `columntransformer__pipeline__polynomialfeatures__degree`. If you want to list the names of all available parameters you can call the `get_params()` method on the model object, e.g. `polyreg_pipe.get_params()` here.

```
[44]: # Create Pipeline for model that includes polynomial expansions of bmi
num_pre = make_pipeline(
    StandardScaler(),
    PolynomialFeatures(include_bias=False)
)

preprocessor = make_column_transformer(
    (OneHotEncoder(drop=np.array(['no']))), ['smoker']), # encode 'smoker' column
    (num_pre, ['bmi']), # polynomial features of 'bmi' column
    verbose_feature_names_out=False)

# Create the new pipeline
polyreg_pipe = make_pipeline(
    preprocessor,
    LinearRegression(), # add step for polynomial features
)

# To see the names of the parameters that can be tuned
# polyreg_pipe.get_params() #names of all parameters
# polyreg_pipe['columntransformer'].get_params() #names of parameters for the
↳preprocessor
```

```
[45]: # Define parameter grid for degree of polynomial features
parameters = {
    'columntransformer__pipeline__polynomialfeatures__degree': np.arange(1,10,1)
}

# Define cross-validation scheme
kf = KFold(n_splits = 5, shuffle = True, random_state=rng)
```

```
# Grid search with cross-validation
grid_search = GridSearchCV(polyreg_pipe, parameters, cv = kf,
                           scoring = 'neg_mean_squared_error',
                           ↪return_train_score=True).fit(X_train, y_train)
```

The above code goes through the process of fitting all 5×9 models as well as storing and ranking the results for the requested scoring metric(s). Note that here we have used `neg_mean_squared_error` as our scoring metric which returns the **negative of the mean squared error**. For more on metrics of regression models, please see: https://scikit-learn.org/stable/modules/model_evaluation.html#regression-metrics

- As the name implies this returns the negative of the usual fit metric, this is because sklearn expects to always optimize for the maximum of a score and the model with the largest negative MSE will therefore be the “best”.
- In this workshop we have used MSE as a metric for testing our models. This metric is entirely equivalent to the root mean squared error for purposes of ranking / ordering models (as the square root is a monotonic transformation).
- Sometimes the RMSE is preferred as it is more interpretable, because it has the same units as y .

Once all of the submodels are fit, we can determine the optimal hyperparameter value by accessing the object's `best_*` attributes,

```
[46]: print("best index: ", grid_search.best_index_)
      print("best param: ", grid_search.best_params_)
      print("best score: ", grid_search.best_score_)
```

```
best index: 2
best param: {'columntransformer__pipeline__polynomialfeatures__degree': 3}
best score: -51205617.18188197
```

The best estimator is stored in the `.best_estimator` attribute. By default, after this model is found, it is retrained on all training data points.

```
[47]: grid_search.best_estimator_[0].coef_
```

```
[47]: array([22996.56516622, 3008.02654513, -395.36430999, -147.96929141])
```

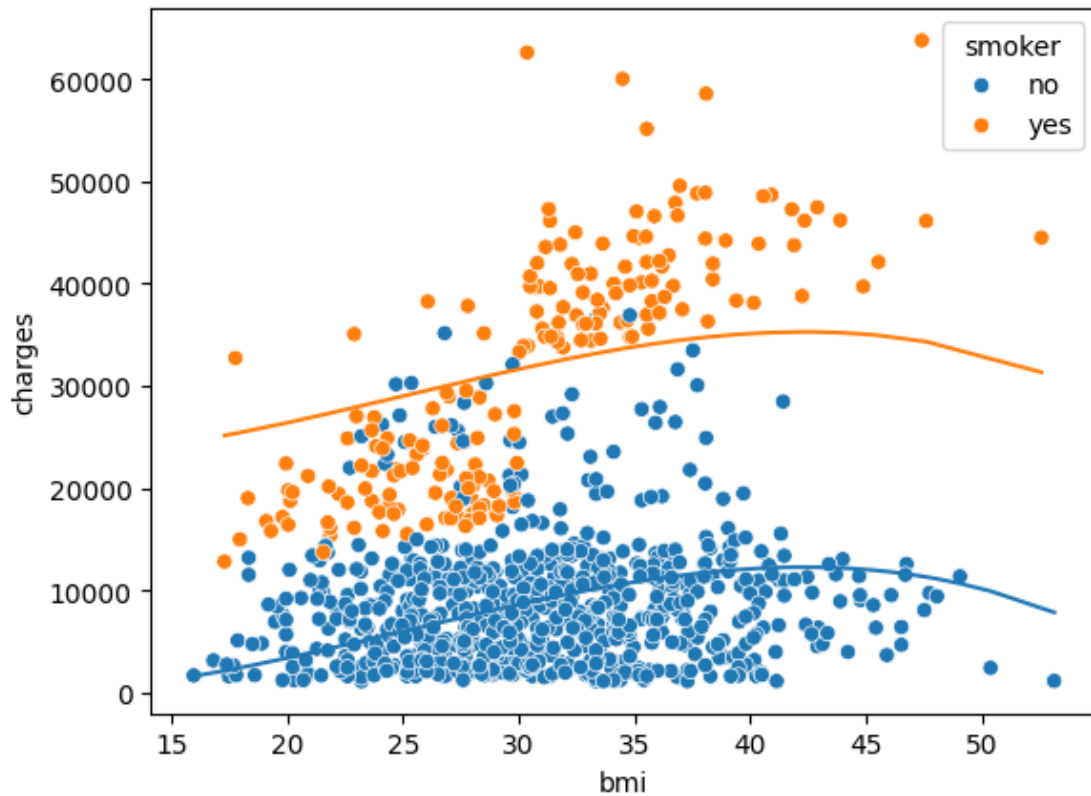
```
[48]: # Extract the names of the features and coefficients
w = [grid_search.best_estimator_[0].intercept_] + grid_search.
     ↪best_estimator_[0].coef_.tolist()
feature_names = ['intercept'] + list(grid_search.best_estimator_[0].
     ↪get_feature_names_out())
w_df = pd.DataFrame({'feature': feature_names, 'coefficient': w})
w_df
```



```
[48]:      feature    coefficient
0  intercept  9010.405147
1  smoker_yes 22996.565166
2      bmi    3008.026545
3     bmi^2   -395.364310
4     bmi^3   -147.969291
```

```
[49]: # Compute fitted values
yhat =grid_search.predict(X_train)

# Plot fitted values
ax = sns.scatterplot(x = X_train.bmi, y = y_train, hue = X_train.smoker)
sns.lineplot(x = X_train.bmi, y = yhat, hue = X_train.smoker, ax=ax, legend =_
↪False)
ax.set(ylabel='charges')
plt.show()
```



The cross-validated scores are stored in the attribute `cv_results_`. This contains a number of results related to the grid search and cross-validation. We can convert it into a pandas data frame to view the results.

```
[50]: cv_results = pd.DataFrame(grid_search.cv_results_)
      cv_results
```

```
[50]:   mean_fit_time  std_fit_time  mean_score_time  std_score_time  \
0      0.003570    0.001215      0.001666      0.000145
1      0.003252    0.000088      0.001643      0.000091
2      0.003428    0.000582      0.001583      0.000073
3      0.003016    0.000061      0.001556      0.000054
4      0.002932    0.000070      0.001521      0.000016
5      0.002881    0.000054      0.001530      0.000018
6      0.002938    0.000058      0.001564      0.000070
7      0.002954    0.000073      0.001520      0.000033
8      0.002887    0.000058      0.001534      0.000029
```

```
   param_columntransformer__pipeline__polynomialfeatures__degree  \
0                                                                    1
1                                                                    2
2                                                                    3
3                                                                    4
4                                                                    5
5                                                                    6
6                                                                    7
7                                                                    8
8                                                                    9
```

```
   params  split0_test_score  \
0 {'columntransformer__pipeline__polynomialfeatu... -4.953120e+07
1 {'columntransformer__pipeline__polynomialfeatu... -4.863959e+07
2 {'columntransformer__pipeline__polynomialfeatu... -4.849697e+07
3 {'columntransformer__pipeline__polynomialfeatu... -4.846416e+07
4 {'columntransformer__pipeline__polynomialfeatu... -4.831336e+07
5 {'columntransformer__pipeline__polynomialfeatu... -4.808782e+07
6 {'columntransformer__pipeline__polynomialfeatu... -4.817562e+07
7 {'columntransformer__pipeline__polynomialfeatu... -4.817024e+07
8 {'columntransformer__pipeline__polynomialfeatu... -4.826592e+07
```

```
   split1_test_score  split2_test_score  split3_test_score  ...  \
0 -5.421318e+07      -4.522664e+07      -5.802686e+07      ...
1 -5.332238e+07      -4.693936e+07      -5.682679e+07      ...
2 -5.323485e+07      -4.696317e+07      -5.677668e+07      ...
3 -5.318416e+07      -4.801395e+07      -5.687521e+07      ...
4 -5.309080e+07      -5.040526e+07      -5.682702e+07      ...
5 -5.313373e+07      -5.008987e+07      -5.815680e+07      ...
6 -5.326807e+07      -5.015651e+07      -5.828802e+07      ...
7 -5.308828e+07      -5.206927e+07      -5.876990e+07      ...
8 -5.320403e+07      -5.375387e+07      -5.884213e+07      ...
```

	mean_test_score	std_test_score	rank_test_score	split0_train_score \
0	-5.166557e+07	4.314726e+06	4	-5.197729e+07
1	-5.128167e+07	3.496021e+06	2	-5.150724e+07
2	-5.120562e+07	3.490302e+06	1	-5.141538e+07
3	-5.140654e+07	3.288286e+06	3	-5.139295e+07
4	-5.184219e+07	2.916572e+06	5	-5.138029e+07
5	-5.195127e+07	3.494678e+06	6	-5.124288e+07
6	-5.203563e+07	3.523907e+06	7	-5.106324e+07
7	-5.244722e+07	3.581987e+06	8	-5.097090e+07
8	-5.283836e+07	3.613367e+06	9	-5.094828e+07

	split1_train_score	split2_train_score	split3_train_score \
0	-5.076974e+07	-5.300751e+07	-4.984967e+07
1	-5.029181e+07	-5.197860e+07	-4.944856e+07
2	-5.018797e+07	-5.185925e+07	-4.932849e+07
3	-5.017128e+07	-5.166660e+07	-4.932628e+07
4	-5.012581e+07	-5.131375e+07	-4.932590e+07
5	-4.993415e+07	-5.130996e+07	-4.889950e+07
6	-4.974739e+07	-5.130986e+07	-4.870798e+07
7	-4.971758e+07	-5.114621e+07	-4.856394e+07
8	-4.969361e+07	-5.109410e+07	-4.856170e+07

	split4_train_score	mean_train_score	std_train_score
0	-5.150686e+07	-5.142221e+07	1.070237e+06
1	-5.095547e+07	-5.083633e+07	8.934741e+05
2	-5.086011e+07	-5.073024e+07	8.966502e+05
3	-5.085550e+07	-5.068252e+07	8.489775e+05
4	-5.077096e+07	-5.058334e+07	7.738649e+05
5	-5.065533e+07	-5.040837e+07	9.024238e+05
6	-5.050278e+07	-5.026625e+07	9.463269e+05
7	-5.045974e+07	-5.017167e+07	9.447169e+05
8	-5.045624e+07	-5.015078e+07	9.330973e+05

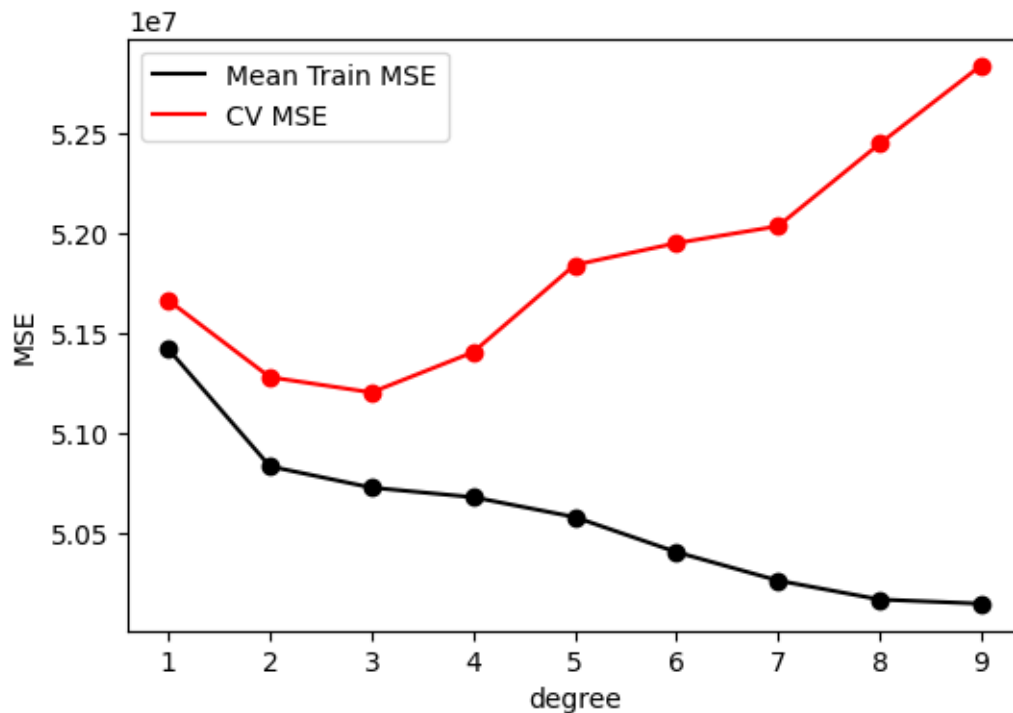
[9 rows x 21 columns]

It is also recommend to plot the CV scores. Although the grid search may report a best value for the parameter corresponding to the maximum CV score (e.g. min CV MSE), if the curve is relatively flat around the minimum, we may prefer the simpler model.

Note in this case, I have also used the option `return_train_score=True` in `GridSearchCV()`, in order to save also the training scores. As expected training MSE decreases when increasing the degree of the polynomial, but the CV MSE has more of a U-shape.

```
[51]: degree = np.arange(1,10,1)
fig, ax = plt.subplots(1,1,figsize=(6,4))
plt.scatter(degree,-grid_search.cv_results_['mean_train_score'], color='k')
plt.plot(degree,-grid_search.cv_results_['mean_train_score'], color='k',
        label='Mean Train MSE')
```

```
plt.scatter(degree,-grid_search.cv_results_['mean_test_score'], color='r')
plt.plot(degree,-grid_search.cv_results_['mean_test_score'], color='r',
        label='CV MSE')
ax.legend()
ax.set_xlabel('degree')
ax.set_ylabel('MSE')
plt.show()
```



6.4.2 Exercise 12 (CORE)

Based on the plot above, would you use the best estimator or choose a different degree? Why?

Try changing `random_state` in the grid search to different value. Does the best estimator still choose the same degree?

The curve is relatively flat around degrees 2 and 3, thus we may prefer the simpler model with degree 2. Moreover, when we change the random state, it sometimes chooses 2 degrees, further supporting the choice of the simpler model.

6.4.3 Exercise 13 (EXTRA)

Try an alternative model of your choice. What have you chosen and why? Are there any parameters to tune?

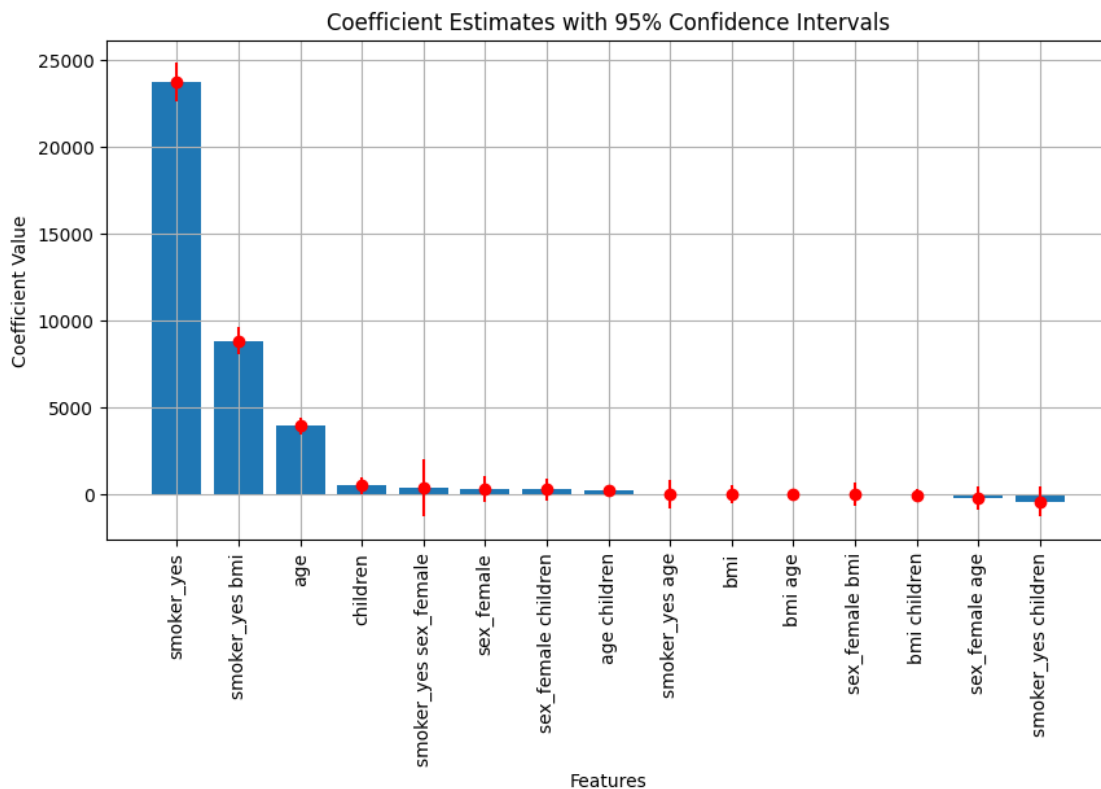
There are many examples that one could try and answers will vary.

Here is an example including the other inputs and all interactions.

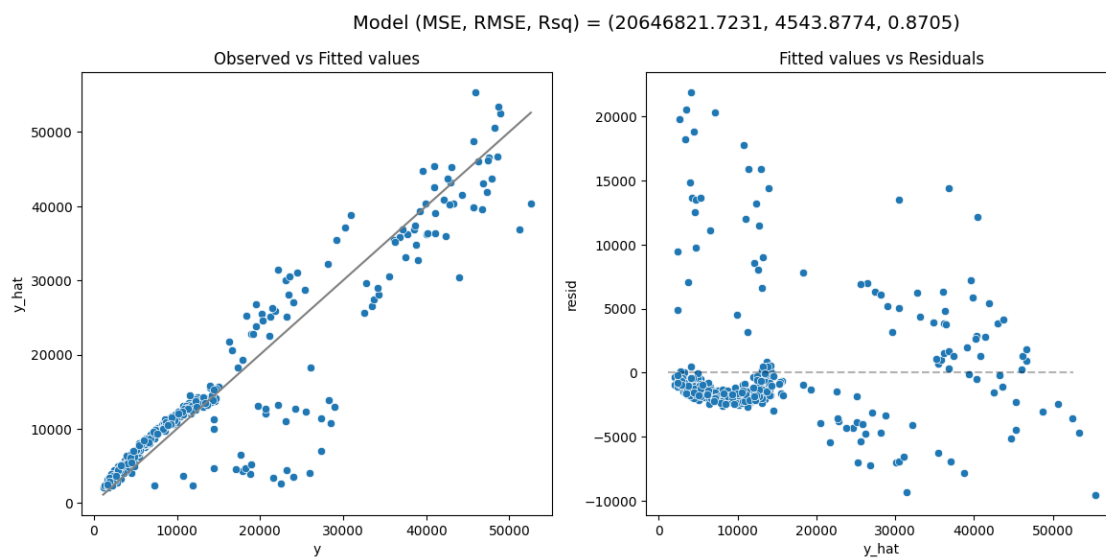
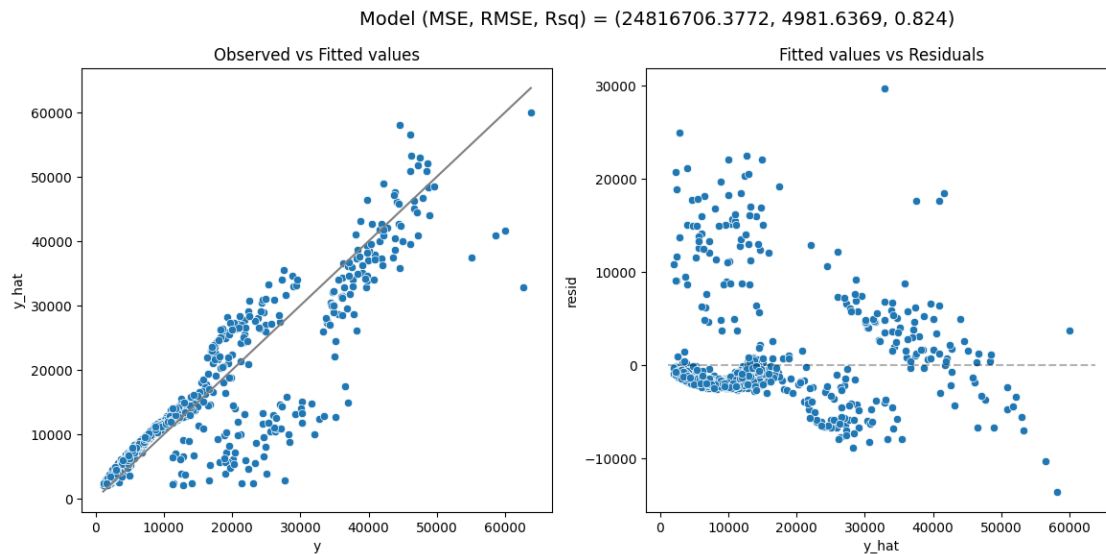
```
[52]: # Create Pipeline for model that includes interactions
preprocessor = make_column_transformer(
    (OneHotEncoder(drop=np.array(['no', 'male'])), ['smoker','sex']), # encode
    ↪ 'smoker' and 'sex' columns
    (StandardScaler(), ['bmi','age', 'children']), # scale 'bmi' column
    verbose_feature_names_out=False)

# Create and fit new pipeline
pipe_all = make_pipeline(
    preprocessor,
    PolynomialFeatures(interaction_only=True,include_bias=False), # add extra
    ↪ step for interaction terms
    LinearRegression()
).fit(X_train, y_train)
```

```
[53]: sd, cov_mat = compute_std_error(pipe_all, X_train, y_train, plot=True,
    ↪ figsize=(10,5))
```



```
[54]: model_fit(pipe_all, X_train, y_train, plot = True)
model_fit(pipe_all, X_test, y_test, plot = True)
```



[54]: (20646821.723093983, 4543.877388651017, 0.8705280555816491)

Here is another example, allowing for different basis coefficients and degree of polynomials for smokers and non-smokers.

```
[55]: # Create Pipeline for model that includes polynomial expansions of bmi
num_pre = make_pipeline(
    StandardScaler()
)
```

```

preprocessor = make_column_transformer(
    (OneHotEncoder(drop=np.array(['no'])), ['smoker']), # encode 'smoker' column
    (num_pre, ['bmi']), # polynomial features of 'bmi' column
    verbose_feature_names_out=False)

# Create the new pipeline
polyreg_pipe2 = make_pipeline(
    preprocessor,
    PolynomialFeatures(interaction_only=True, include_bias=False), # add step
    ↪for interaction terms,
    make_column_transformer(
        ('passthrough', [0]),
        (PolynomialFeatures(include_bias=False), [1]),
        (PolynomialFeatures(include_bias=False), [2]),
        verbose_feature_names_out=False
    ),
    LinearRegression(), # add step for polynomial features
).fit(X_train, y_train)

#polyreg_pipe2[:-1].get_feature_names_out()
#polyreg_pipe2.get_params()

```

```

[56]: # Parameter grid for degrees of polynomial features
parameters = {
    'columntransformer-2__polynomialfeatures-1__degree': np.arange(1,5,1),
    'columntransformer-2__polynomialfeatures-2__degree': np.arange(1,5,1)
}

# Cross-validation scheme
kf = KFold(n_splits = 5, shuffle = True, random_state=rng)

# Grid search with cross-validation
grid_search2 = GridSearchCV(polyreg_pipe2, parameters, cv = kf,
    scoring = 'neg_mean_squared_error',
    ↪return_train_score=True
    ).fit(X_train, y_train)

```

```

[57]: print("best param: ", grid_search2.best_params_)
print("best score: ", grid_search2.best_score_)

```

```

best param: {'columntransformer-2__polynomialfeatures-1__degree': 3,
'columntransformer-2__polynomialfeatures-2__degree': 1}
best score: -39716930.37195696

```

```

[58]: # Print coefficients of the best estimator
w = [grid_search2.best_estimator_[-1].intercept_] + grid_search2.
    ↪best_estimator_[-1].coef_.tolist()

```

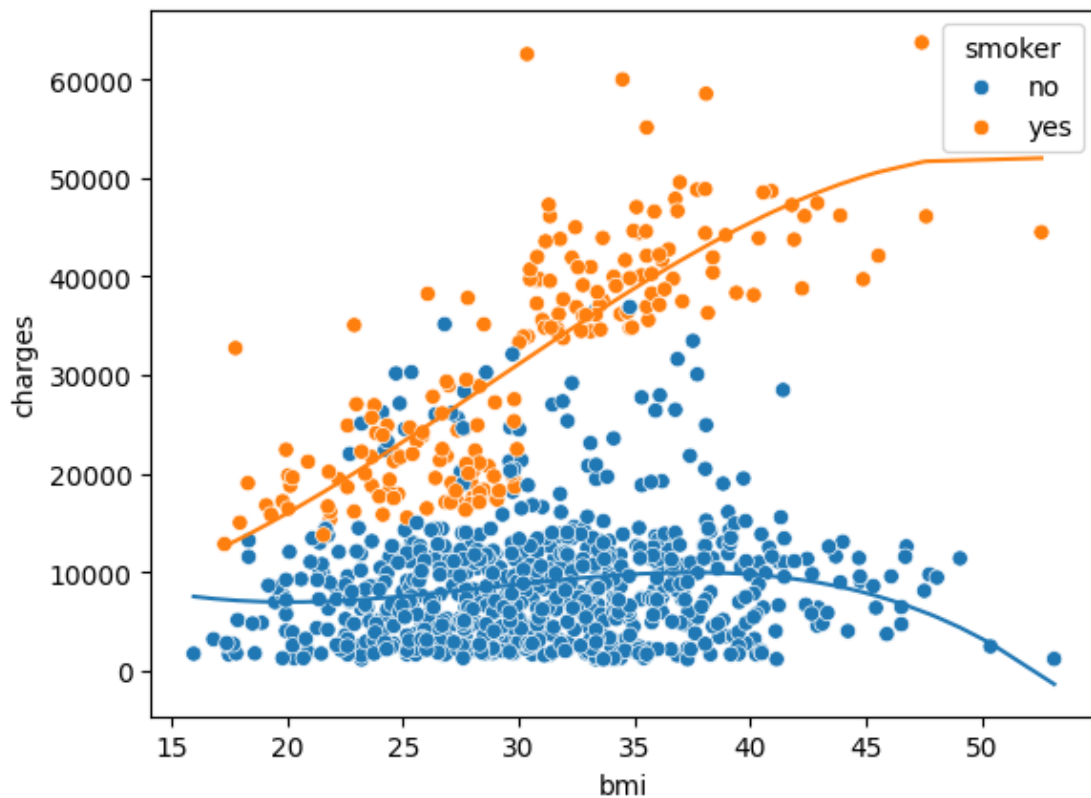
```
feature_names = ['intercept'] + list(grid_search2.best_estimator_[:-1].
    ↪get_feature_names_out())
w_df = pd.DataFrame({'feature': feature_names, 'coefficient': w})
w_df
```

```
[58]:
```

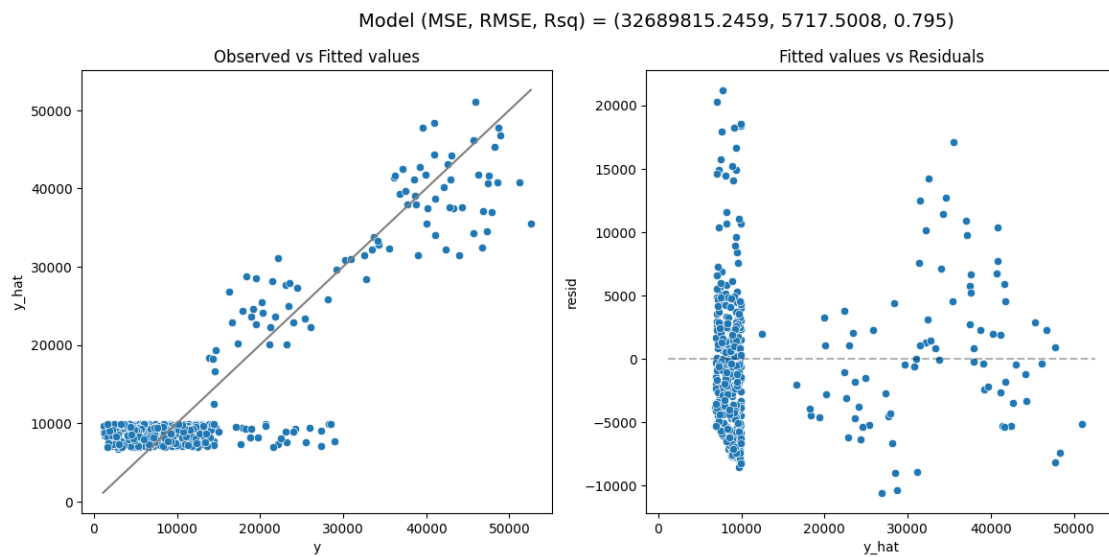
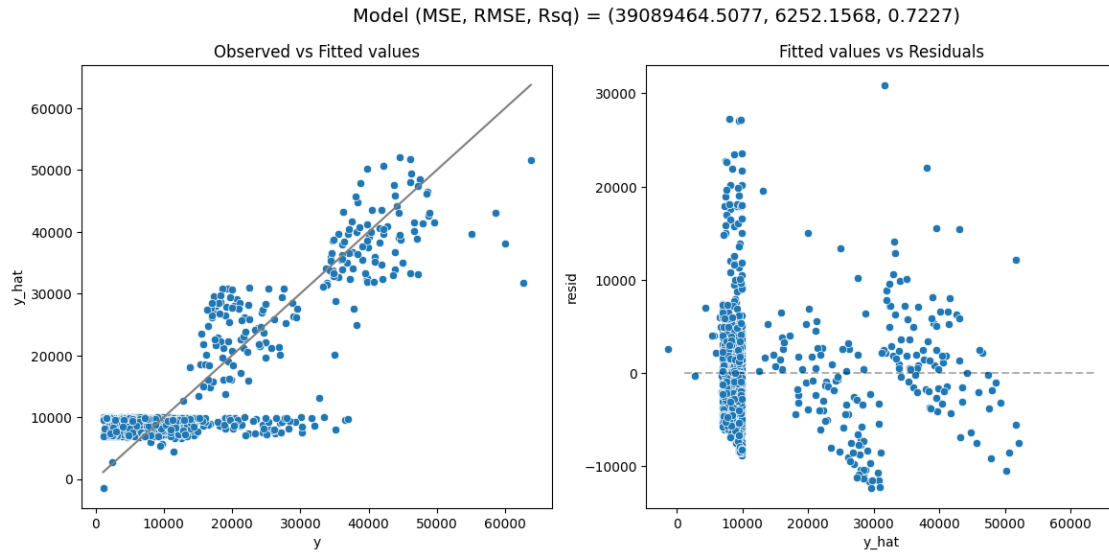
	feature	coefficient
0	intercept	8938.796877
1	smoker_yes	23368.329245
2	bmi	1523.594189
3	bmi ²	-254.532234
4	bmi ³	-270.602085
5	smoker_yes bmi	8317.473433

```
[59]: # Compute fitted values
yhat = grid_search2.predict(X_train)

# Plot fitted values
ax = sns.scatterplot(x = X_train.bmi, y = y_train, hue = X_train.smoker)
sns.lineplot(x = X_train.bmi, y = yhat, hue = X_train.smoker, ax=ax, legend =
    ↪False)
ax.set(ylabel='charges')
plt.show()
```




```
[60]: model_fit(grid_search2.best_estimator_, X_train, y_train, plot = True)
      model_fit(grid_search2.best_estimator_, X_test, y_test, plot = True)
```



```
[60]: (32689815.245917737, 5717.500786700229, 0.7950089365167734)
```

As another example, let's explore using step functions to allow nonlinearity.

```
[61]: # Create Pipeline for model that includes polynomial expansions of bmi
from sklearn.preprocessing import KBinsDiscretizer

num_pre = make_pipeline(
    StandardScaler()
)

preprocessor = make_column_transformer(
    (OneHotEncoder(drop=np.array(['no'])), ['smoker']), # encode 'smoker' column
    (num_pre, ['bmi']), # polynomial features of 'bmi' column
    verbose_feature_names_out=False)

# Create the new pipeline
stepreg_pipe = make_pipeline(
    preprocessor,
    PolynomialFeatures(interaction_only=True, include_bias=False), # add step
    ↪for interaction terms,
    make_column_transformer(
        ('passthrough', [0]),
        (KBinsDiscretizer(strategy='uniform'), [1]),
        (KBinsDiscretizer(strategy='uniform'), [2]),
        verbose_feature_names_out=False
    ),
    LinearRegression(), # add step for polynomial features
).fit(X_train, y_train)

#stepreg_pipe[:-1].get_feature_names_out()
#stepreg_pipe.get_params()
```

```
[62]: # Parameter grid for degrees of polynomial features
parameters = {
    'columntransformer-2__kbinstdiscretizer-1__n_bins': np.arange(2,8,1),
    'columntransformer-2__kbinstdiscretizer-2__n_bins': np.arange(2,8,1)
}

# Cross-validation scheme
kf = KFold(n_splits = 5, shuffle = True, random_state=rng)

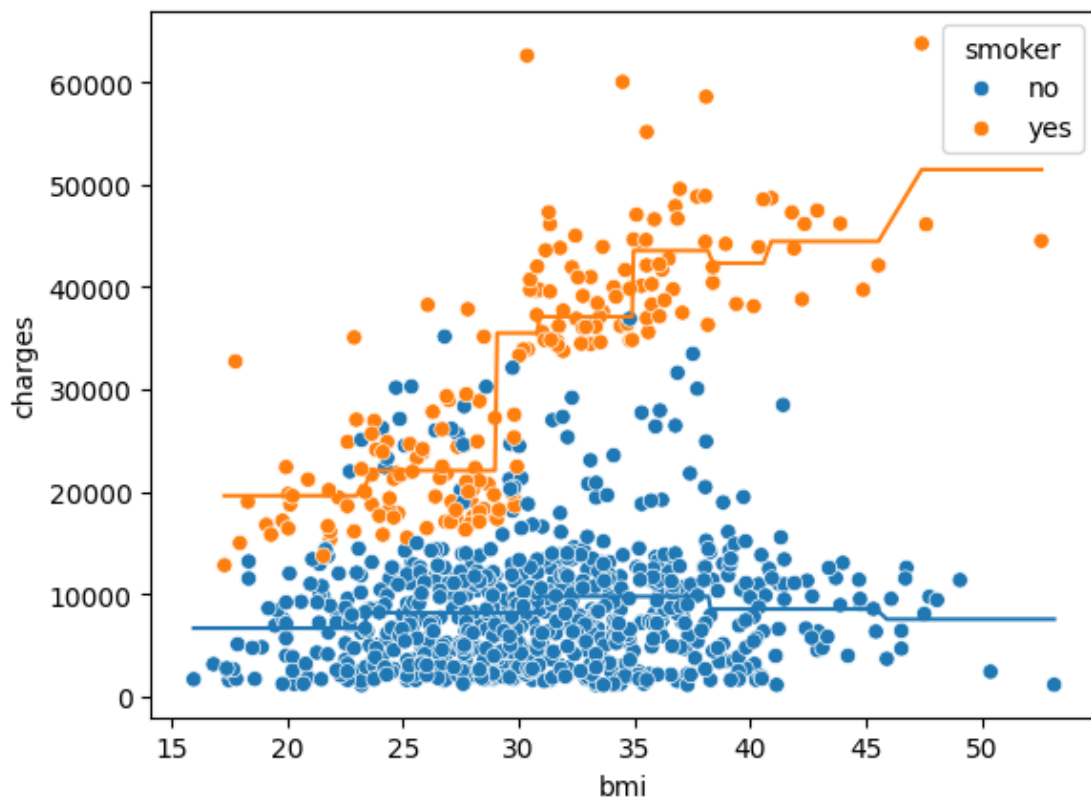
# Grid search with cross-validation
grid_search_step = GridSearchCV(stepreg_pipe, parameters, cv = kf,
                                scoring = 'neg_mean_squared_error',
    ↪return_train_score=True
                                ).fit(X_train, y_train)

print("best param: ", grid_search_step.best_params_)
print("best score: ", grid_search_step.best_score_)
```

```
best param: {'columntransformer-2__kbinstdiscretizer-1__n_bins': 5,
```

```
'columntransformer-2__kbinsdiscretizer-2__n_bins': 6}  
best score: -38133217.963237435
```

```
[63]: # Compute fitted values  
yhat = grid_search_step.predict(X_train)  
  
# Plot fitted values  
ax = sns.scatterplot(x = X_train.bmi, y = y_train, hue = X_train.smoker)  
sns.lineplot(x = X_train.bmi, y = yhat, hue = X_train.smoker, ax=ax, legend =  
↳False)  
ax.set(ylabel='charges')  
plt.show()
```



Let's see another example using splines.

```
[64]: # Create Pipeline for model that includes polynomial expansions of bmi  
from sklearn.preprocessing import SplineTransformer  
  
num_pre = make_pipeline(  
    StandardScaler()  
)
```

```

preprocessor = make_column_transformer(
    (OneHotEncoder(drop=np.array(['no'])), ['smoker']), # encode 'smoker' column
    (num_pre, ['bmi']), # polynomial features of 'bmi' column
    verbose_feature_names_out=False)

# Create the new pipeline
splinereg_pipe = make_pipeline(
    preprocessor,
    PolynomialFeatures(interaction_only=True, include_bias=False), # add step
    ↪for interaction terms,
    make_column_transformer(
        ('passthrough', [0]),
        (SplineTransformer(), [1]),
        (SplineTransformer(), [2]),
        verbose_feature_names_out=False
    ),
    LinearRegression(), # add step for polynomial features
).fit(X_train, y_train)

#splinereg_pipe[:-1].get_feature_names_out()
#splinereg_pipe.get_params()

```

```

[65]: # Parameter grid for degrees of polynomial features
parameters = {
    'columntransformer-2__splinetransformer-1__n_knots': np.arange(2,6,1),
    'columntransformer-2__splinetransformer-2__n_knots': np.arange(2,6,1)
}

# Cross-validation scheme
kf = KFold(n_splits = 5, shuffle = True, random_state=rng)

# Grid search with cross-validation
grid_search_spline = GridSearchCV(splinereg_pipe, parameters, cv = kf,
    ↪scoring = 'neg_mean_squared_error',
    ↪return_train_score=True
    ).fit(X_train, y_train)

print("best param: ", grid_search_spline.best_params_)
print("best score: ", grid_search_spline.best_score_)

```

```

best param: {'columntransformer-2__splinetransformer-1__n_knots': 4,
'columntransformer-2__splinetransformer-2__n_knots': 5}
best score: -38187884.26085524

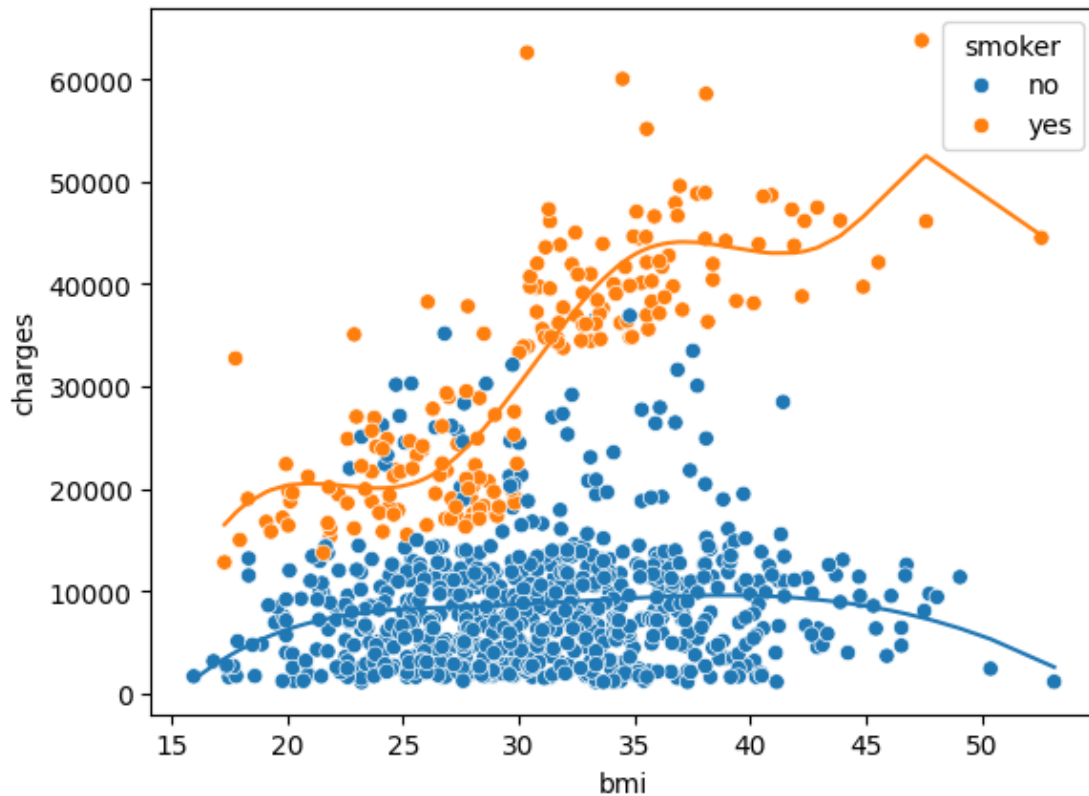
```

```

[66]: # Compute fitted values
yhat = grid_search_spline.predict(X_train)

```

```
# Plot fitted values
ax = sns.scatterplot(x = X_train.bmi, y = y_train, hue = X_train.smoker)
sns.lineplot(x = X_train.bmi, y = yhat, hue = X_train.smoker, ax=ax, legend = False)
ax.set(ylabel='charges')
plt.show()
```



6.4.4 Further resources

- About common pitfalls and interpreting coefficients:

https://scikit-learn.org/stable/auto_examples/inspection/plot_linear_model_coefficient_interpretation.html#

7 Competing the Worksheet

At this point you have hopefully been able to complete all the CORE exercises and attempted the EXTRA ones. Now is a good time to check the reproducibility of this document by restarting the notebook's kernel and rerunning all cells in order.

Before generating the PDF, please **change 'Student 1' and 'Student 2' at the top of the notebook to include your name(s)**.

Once that is done and you are happy with everything, you can then run the following cell to generate your PDF.

```
[67]: # !jupyter nbconvert --to pdf mlp_week04_key.ipynb
```